

RFC: Shared Chunk Cache Design

Clay Carper clay.carper@lifeboat.llc
John Mainzer john.mainzer@lifeboat.llc

This document outlines the design of the Shared Chunk Cache (SCC). Initially, SCC will support I/O operations for structured chunks—a newly introduced HDF5 storage paradigm for sparse and variable-length data. In the future, it may replace the current chunk cache implementation, extending its functionality to support dense chunks as well. The cache is also designed to enable multi-threaded access in the future.

1	Introduction	4
2	Overview of Structures.....	4
2.1.	H5SC_chunk_key_t – Chunk Indexing.....	6
2.2.	H5SC_chunk_t – Individual Cache Entry Structure	6
2.3.	H5SC_io_info_t – Structure Used for an I/O Request on a Dataset.....	9
2.4.	H5SC_io_sel_chunk_t – Callback-Related Chunk Information for I/O Ops.....	10
2.5.	H5SC_dset_header_t – Dataset-Based Chunk Tracking	12
2.6.	H5SC_stats_t – Per-File Global Cache Statistics	14
2.7.	H5SC__cache_config_t – Shared Chunk Cache Configuration Dataset-Based	16
2.8.	H5SC_t – Per-File Global Cache Management.....	16
2.9.	H5SC_exhausted_list_t – List for Eviction-Related Dataset Exhaustion.....	18
3	Theory of Operations	19
3.1.	File Open	19
3.2.	Dataset Open	19
3.3.	Dataset Flush.....	20
3.4.	Dataset Erase	20
3.5.	Applying Filters Through the SCC	21
3.6.	Dataset Change Extent	22
3.7.	Dataset Close.....	23
3.8.	Writing Data Through the SCC	24
3.9.	Reading Data Through the SCC	24
3.10.	Flush the Cache	25
3.11.	File Close	25
4	Detailed Component Discussions.....	26
4.1.	Chunk Tracking Through the Hash Table.....	26
4.2.	Eviction.....	26
4.2.1	Additional Eviction Strategies	27
4.3.	Batching.....	27
4.4.	Public Configuration Structure	28
4.5.	Filter Configuration and Usage.....	28
4.5.1	GZIP / DEFLATE Filtering.....	29
4.5.2	SZIP Filtering.....	29

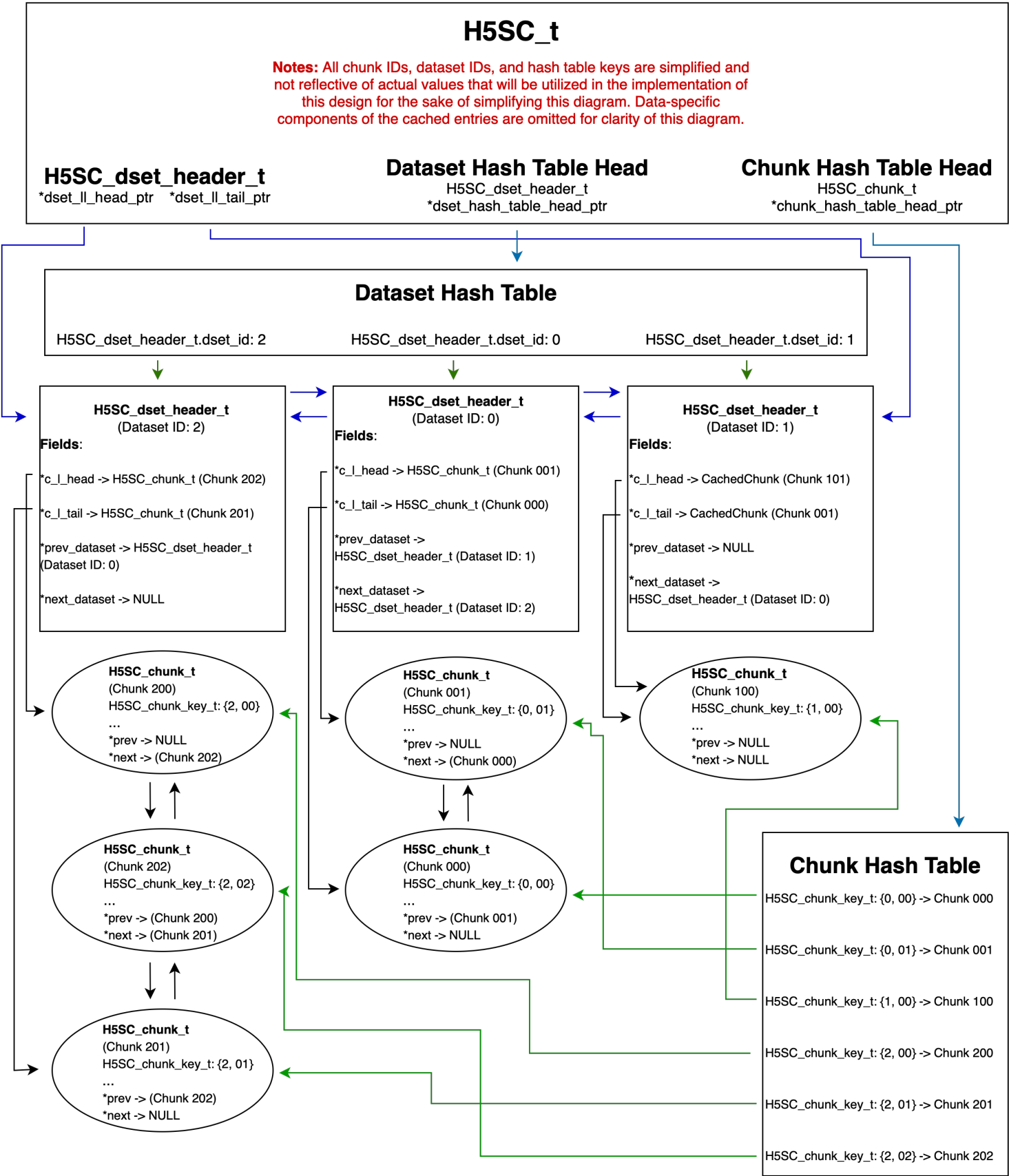
4.5.3	Mixed Filter Pipelines	30
5	Top Level API	31
6	SCC Internal Functions	34
6.1.	Hash table specific functions.....	34
6.2.	DLL specific functions	35
6.3.	H5SC_chunk_t specific functions	36
6.4.	I/O specific functions.....	37
6.5.	Eviction specific functions	38
6.6.	Size accounting functions.....	39
6.7.	Extensible dimension-related functions.....	40
6.8.	Statistics-specific functions	42
7	Layout Callbacks	43
7.1.1	Callback struct	49
8	Call Tree Diagrams.....	49
8.1.	Reading data through H5Dread():	49
8.1.1	Data Read Call Tree by Subsystem:	50
8.2.	Writing data through H5Dwrite():	51
8.2.1	Data Write Call Tree by Subsystem:	52
8.3.	Supporting Initialization and Dataset Setup.....	53
8.4.	Eviction and Flush Subsystems.....	53
8.5.	H5Derase() Functionality.....	53
8.6.	H5Dset_extent Functionality.....	54
9	Test Coverage Summary.....	56
10	Multi-threaded support	57
10.1.	Multi-threaded operations in SCC.....	57
10.2.	Additional code for supporting multi-thread operations in SCC	58
11	Acknowledgement	58
12	Appendix A: SCC Version 0 Implementation Description	58
13	References.....	60

1 Introduction

This document outlines the design for the Shared Chunk Cache (SCC). The new cache not only provides support for structured chunks, a new data storage introduced to HDF5, but serves as an opportunity to address additional requirements [1] and known limitations of the existing chunk cache. For example, a single cache will be utilized across all datasets in a single HDF5 file, enabling the SCC to utilize a minimalistic hash table, while also utilizing portions of memory that would have otherwise been inaccessible due to the nature of the previously utilized cache-per-dataset approach. The remainder of this document will focus on the design of the SCC, including the proposed structures, how they satisfy the design requirements, and how various procedures will utilize these components.

2 Overview of Structures

To support and inform the underlying design decisions, this document will first focus on the underlying data structures which have been developed for the initial single-thread-enabled version of the SCC. Additionally, considerations for future multi-thread support outlined in section 10 have been influential in this development. Further, the diagram below provides an illustration of the structures discussed in this document.



2.1. H5SC_chunk_key_t – Chunk Indexing

```

/*****
 *
 * Structure: H5SC_chunk_key_t
 *
 * Info
 *
 * The structured stores a unique 128-bit key for a chunk. The key is derived
 * from the dataset's object header address in the file and the chunk's
 * serialized logical coordinates using an LSB-first bit interleaving schema.
 *
 * Fields
 *
 * uint64_t low_half - The bottom 64-bits of the 128-bit integer key.
 *
 * uint64_t high_half - The top 64-bits of the 128-bit integer key.
 *
 * Example: Assume 64-bit dataset object header address = 610 or ...01102,
 *          and 64-bit chunk logical coordinate = 510 or ...01012.
 *          LSB-first bit interleaving yields to 0,1,1,0,1,1 → 01101100...2
 *          with remaining zeros omitted for clarity. The result of
 *          the operation is stored in the two 64-bit integer fields
 *          high_half = 01101100...2; low_half = 00000000...2;
 *
 *****/

struct H5SC_chunk_key_t {
    uint64_t high_half;
    uint64_t low_half;
} H5SC_chunk_key_t;

```

Design Notes

No major notes; this structure does what it says on the tin.

2.2. H5SC_chunk_t – Individual Cache Entry Structure

```

/*****
 *
 * Structure: H5SC_chunk_t
 *
 * Info
 *
 * The structure represents an individual chunk entry in SCC; it is an entry
 * in the chunk hash table.
 *
 * The structure stores the chunk key, pointers to in-memory and on disk chunk
 * data, chunk size, state flags, and pointers necessary to track the chunk as
 * a member of the associated dataset's chunk LRU list. A hash handle is
 * included to make the structure hashable.
 *
 * Fields
 *
 * uint64_t magic - An integer value that is set by internal functions that

```

```

*         create an instance of H5SC_chunk_t structure. Expected to be set to be
*         1396917064. This value is used to verify that an instance of a
*         H5SC_chunk_t structure was created properly.
*
* H5SC_chunk_key_t data_key - The unique key associated with the cached
*         chunk. It is primarily used for chunk lookups through the chunk-
*         focused hash table and operations that require internal chunk index
*         updates.
*
* void *chunk_obj - Pointer to the decoded chunk object, which is an
*         H5D_chunk_cache_mem_t structure. Necessary for callback operations.
*
* void *udata - Pointer to the chunk udata, which is an H5D_chunk_ud_t
*         structure. Necessary for callback operations.
*
* unsigned ndims - Number of valid dims defined in the scaled coordinates for
*         this chunk. The default value is 0, a convenient invalid value.
*
* hsize_t chunk_log_coord - The linearized, dataset-relative chunk coordinate,
*         referred to as the chunk logical coordinate (index). This value is
*         computed from the dataset dimensions, chunk dimensions, and element
*         position. It is computed when a H5SC_chunk_t structure is added to the
*         SCC and is used as one of the components for the chunk key
*         (H5SC_chunk_key_t). This value is stored as a reference for checking
*         whether a chunk should be reindexed when a dataset contains one or
*         more extensible dimensions.
*
* hsize_t scaled[H5S_MAX_RANK] - The scaled coordinates associated with this
*         chunk (essentially the offset divided by the chunk dimension (for each
*         rank)). This array is set during the initialization of an I/O request
*         and is used for cache flush procedures and callback operations.
*
* haddr_t disk_addr - The on-disk address of the chunk. Set to HADDR_UNDEF
*         when the chunk is created. Necessary for cache hits to be processed
*         correctly.
*
* size_t cached_chunk_size - The size of the in-memory data buffer after
*         decoding the chunk and the application of any required filters. This
*         field is updated when the in-memory data is altered. The default
*         value is set to 0.
*
* size_t disk_nbytes - The last known size of the on disk data buffer prior to
*         applying filters or decoding. It is obtained using the
*         H5D__struct_chunk_lookup callback. The default value is set to 0.
*
* size_t chunk_counter - A priority counter primarily used with eviction
*         policies. It is set to 0 by default. In the future, it may be used to
*         identify eviction candidates based on access history or other encoded
*         states. For example, during eviction candidate selection, if the
*         partial_io flag is set to True, this counter would be incremented to a
*         value of 1 to indicate that it has been flagged for a second pass
*         through the LRU list.
*
* H5SC_tag_t last_tag - Used to indicate what the most recent internal
*         operation was that interacted with this chunk. Set to H5SC_TAG_CREATE
*         when a H5SC_chunk_t structure is created. See the section defining
*         H5SC_tag_t for possible values.

```

```

*
* bool dirty_flag - A flag to indicate when an in-memory chunk buffer has been
*                   modified. A value of True indicates a chunk buffer is dirty. This flag
*                   is set when a chunk is created/modified/resized. Set to False by
*                   default.
*
* bool partial_io - A flag to indicate if a partial read or write operation
*                   has been done on this chunk (i.e., the intersection of the selection
*                   describing this chunk and the selection on the dataset resulted in
*                   less than the whole chunk being selected). It is used to track I/O
*                   states and inform eviction policies. This field is set to false by
*                   default.
*
* struct H5SC_chunk_t *prev_ptr - The pointer to the previous node in the
*                                  dataset's LRU list. It is Null if this chunk is at the head of the LRU
*                                  list or if the LRU list has a single element.
*
* struct H5SC_chunk_t *next_ptr - The pointer to the next node in the
*                                  dataset's LRU list. It is Null if this chunk is at the tail of the LRU
*                                  list or if the LRU list has a single element.
*
* UT_hash_handle hval_chunk - A hash handle is required for this structure to
*                              be hashable by UTHash; it is used to track this chunk in the SCC chunk
*                              hash table. Set to Null by default.
*
*****/

struct H5SC_chunk_t {
    uint64_t          magic;
    H5SC_chunk_key_t  data_key;
    void              *chunk_obj;
    void              *udata;
    unsigned          ndims;
    hsize_t           chunk_log_coord;
    hsize_t           scaled[H5S_MAX_RANK];
    haddr_t           disk_addr;
    size_t            cached_chunk_size;
    haddr_t           disk_nbytes;
    size_t            disk_chunk_size;
    size_t            chunk_counter;
    H5SC_tag_t        last_tag;
    bool              dirty_flag;
    bool              partial_IO;
    H5SC_chunk_t      *prev_ptr;
    H5SC_chunk_t      *next_ptr;
    UT_hash_handle     hval_chunk;
} H5SC_chunk_t;

```

Design Notes

This structure is created for a chunk within an I/O request during the initialization step (`H5SC__io_info_init()`). Upon creation, the chunk is treated as a “placeholder chunk”; that is, a chunk that does not yet contain any cached data. As a placeholder, this chunk is protected from eviction until it has been properly initialized. This is done to ensure that a chunk present within an I/O request is not prematurely evicted before it can be utilized. Once a chunk has been updated, it is treated as a “resident chunk”. Resident chunks are eligible for eviction candidate selection. It is also

worth noting that this structure contains some fields that are similar to `H5SC_io_sel_chunk_t`. For Version 1, this has largely been accepted as necessary. However, it may be worth considering merging these structures.

2.3. `H5SC_io_info_t` – Structure Used for an I/O Request on a Dataset

```

/*****
 *
 * Structure: H5SC_io_info_t
 *
 * Info
 *
 * The structure represents the state associated with a dataset I/O request
 * in SCC. It encapsulates the set of chunks participating in the operation
 * and provides bookkeeping fields to manage iteration and storage of those
 * selected chunks.
 *
 * The structure primarily serves as a container for chunk selection results
 * produced during I/O setup. It is used to track which chunks are involved in
 * an I/O operation, the starting position (used for batching), how many chunks
 * are present in the request, and how large sel_chunks currently is (in
 * bytes). These fields are subsequently consumed by I/O execution paths and
 * chunk processing routines.
 *
 * Fields
 *
 * H5SC_io_sel_chunk_t *sel_chunks - Pointer to a vector of structures
 * describing the chunks selected for the current I/O operation. Each
 * element contains per-chunk selection and metadata required to
 * execute I/O on that chunk. Default size of 128 and is dependent on
 * the size of sel_chunk_alloc.
 *
 * size_t sel_start - The starting index within sel_chunks for processing.
 * This field is typically used to support batched processing of selected
 * chunks.
 *
 * size_t num_sel_chunks - The number of valid chunk entries currently
 * stored in sel_chunks. This represents the number of chunks
 * participating in the I/O request.
 *
 * size_t sel_chunks_allocated - The total number of elements allocated for
 * sel_chunks. This value defines the current capacity of the array
 * and is used to determine when reallocation is required as more
 * chunks are selected. This value is 128 by default and increases by
 * multiples of 2 as necessary.
 *****/

struct H5SC_io_info_t {
    H5SC_io_sel_chunk_t *sel_chunks;
    size_t sel_start;
    size_t num_sel_chunks;
    size_t sel_chunks_allocated;
};

```

Design Notes

2.4. H5SC_io_sel_chunk_t – Callback-Related Chunk Information for I/O Ops

```

/*****
 *
 * Structure: H5SC_io_sel_chunk_t
 *
 * Info
 *
 * The structure represents a single chunk participating in a dataset I/O
 * request in SCC. It encapsulates all per-chunk state required to map
 * between dataset coordinates, chunk indexing, memory/file dataspace,
 * and any associated cached chunk entry used during I/O execution.
 *
 * The structure serves as the fundamental unit of work for chunked I/O
 * operations. It binds together spatial information (coordinates and
 * scaled indices), cache state, and dataspace selections needed to
 * correctly perform partial or full reads/writes on an individual chunk.
 * Lookup-related fields are included to support lookup-based behavior.
 *
 * Fields
 *
 * H5D_dset_io_info_t *dset_info - Pointer to the dataset I/O context for
 * the dataset this chunk belongs to. Provides access to layout,
 * filter pipeline, and other dataset-level information required
 * for I/O execution.
 *
 * hsize_t coords[H5S_MAX_RANK] - The absolute coordinates of the chunk
 * within the dataset's logical space. These correspond to the
 * chunk's offset in each dimension.
 *
 * hsize_t scaled[H5S_MAX_RANK] - The scaled chunk coordinates, computed
 * by dividing coords by the chunk dimensions for each rank. These
 * values are used for chunk indexing, hashing, and cache lookup
 * operations.
 *
 * hsize_t chk_log_coord - The computed logical coordinate of the chunk,
 * uniquely identifying the chunk within the dataset's chunk index
 * space. This is typically derived from the scaled coordinates.
 *
 * H5SC_chunk_t *cached_chunk - Pointer to the corresponding SCC cache
 * entry for this chunk. This is populated when the chunk is found
 * or inserted into the cache and is NULL if no cache entry has
 * been associated yet.
 *
 * bool lookup_valid - Indicates whether the lookup-related fields
 * contain valid information for this chunk.
 *
 * haddr_t lookup_addr - The on-disk address obtained from lookup
 * operations.
 *
 * hsize_t lookup_disk_nbytes - The size of the chunk on disk as
 * returned by lookup operations.
 *
 * hsize_t lookup_defined_values_size - Size of defined values metadata
 * associated with the chunk from lookup operations.
 *
 * size_t lookup_size_hint - Size hint used to optimize lookup-related
 * operations.
 */

```

```

*
*   size_t lookup_defined_values_size_hint - Size hint for defined
*       values metadata used during lookup.
*
*   void *lookup_udata - Pointer to user data structure used during
*       lookup callbacks.
*
*   H5S_t *file_space - Dataspace describing the selection within the
*       chunk in file coordinates. The extent matches the chunk
*       dimensions, and the selection represents the portion of the
*       chunk involved in the I/O operation.
*
*   H5S_t *mem_space - Dataspace describing the selection within the
*       user's memory buffer corresponding to this chunk. The extent
*       matches the overall memory dataspace, with the selection
*       restricted to the elements mapped to this chunk.
*
*   bool file_space_shared - Flag indicating whether file_space is shared with
*       another owner. This boolean is false by default and indicates that the
*       selection was specific for this chunk. Set to true when the filespace
*       is reused across multiple chunk-selection records.
*
*   bool mem_space_shared - Flag indicating whether mem_space is shared with
*       another owner. This boolean is false by default and indicates that the
*       selection was specific for this chunk. Set to true when the memspace
*       is reused across multiple chunk-selection records.
*
*   H5_flexible_const_ptr_t buf - Pointer to the user-provided memory
*       buffer used for I/O. This buffer is used as the source or
*       destination for data transferred between memory and the chunk.
*
*****/

struct H5SC_io_sel_chunk_t {
    H5D_dset_io_info_t    *dset_info;
    hsize_t               coords[H5S_MAX_RANK];
    hsize_t               scaled[H5S_MAX_RANK];
    hsize_t               chk_log_coord;
    H5SC_chunk_t          *cached_chunk;
    bool                  lookup_valid;
    haddr_t               lookup_addr;
    hsize_t               lookup_disk_nbytes;
    hsize_t               lookup_defined_values_size;
    size_t                lookup_size_hint;
    size_t                lookup_defined_values_size_hint;
    void                  *lookup_udata;
    H5S_t                 *file_space;
    H5S_t                 *mem_space;
    bool                  file_space_shared;
    bool                  mem_space_shared;
    H5_flexible_const_ptr_t buf;
};

```

Design Notes

This structure is a hold-over from the Version 0 functionality. It will likely be combined into `H5SC_chunk_t`, however this has been reserved as an optimization for the time being.

2.5. H5SC_dset_header_t – Dataset-Based Chunk Tracking

```

/*****
 *
 * Structure: H5SC_dset_header_t
 *
 * Info
 *
 * The header structure for a dataset with data held in the shared chunk cache.
 * It tracks the dataset identity (represented by the dataset object header
 * address), the minimum amount of chunk data that should be maintained in this
 * dataset (in bytes), accounts for the total size of cached chunks. The
 * structure contains pointers to the head and tail of the LRU list for the
 * cached chunks in this dataset, pointers to the next and previous datasets in
 * the dataset LRU list, and pointers which are necessary for eviction
 * management.
 *
 * Fields
 *
 * uint64_t magic - Value set when a H5SC_dset_header_t structure is created.
 * This value is used for debugging to ensure that the structure is
 * created/destroyed consistently.
 *
 * haddr_t dset_addr - The 64-bit address of the object header for this
 * dataset. This field is used as the key for the dataset hash table.
 * It is set to 0 by default.
 *
 * size_t min_dset_size - Represents the minimum number of bytes to
 * remain cached for this dataset under typical memory constraints. The
 * default size is 10 MB and is user-configurable.
 *
 * size_t chunk_lru_len - The number of chunks present within the SCC. This
 * value is used as the running count needed for DLL operations. Set to 0
 * by default.
 *
 * size_t curr_dset_size - The current amount of chunk data, in bytes *
 *
 * associated with this dataset within the cache (i.e., is a sum of
 * current chunk sizes). This quantity is updated when a chunk is added,
 * removed, or resized. This field contributes to total cache
 * utilization tracked in H5SC_t. It is set to 0 by default.
 *
 * H5SC_tag_t last_tag - An enum-typed tag used during debugging. Tags are
 * updated near the end of a routine. H5SC_tag_t values do not
 * differentiate between chunks and datasets.
 *
 * H5SC_io_info_t *io_info - Structure used for an I/O request that includes
 * chunks from this dataset. This structure is populated by
 * H5SC_io_info_init() and is used by H5SC_read() and H5SC_write().
 *
 * bool resize_in_progress - A flag used to indicate that a resize operation is
 * being conducted on this dataset. A value of True is set if dataset is
 * being resized. While true, any internal SCC
 * procedures and any I/O requests for this dataset should be interrupted
 * until the resize operation is completed. It is set to false by
 * default.
 *
 * bool evict_exhausted - A flag used to indicate that this dataset has

```

```

*      nominated as many eviction candidates as it is able to and should not
*      be overlooked for future eviction candidate selections during this I/O
*      cycle. During eviction candidate selection, this field is set to True
*      if H5SC_chunk_t elements have been removed and the min_dset_size has
*      been reached. When this field is True, this dataset header is removed
*      from the dataset LRU list and is added to an exhausted dataset list
*      through utilizing the associated pointers (*next_exhausted_ptr and
*      *prev_exhausted_ptr). This field is set to False by default.
*
*      H5D_t *dset - Pointer to the dataset; included for debugging convenience.
*
*      H5SC_chunk_t *lru_head_ptr - The pointer to the head of this dataset's chunk
*      LRU list (indicating it is the most recently used chunk). Set to
*      Null by default, when the chunk LRU list is empty, or when the chunk
*      LRU list contains a single element.
*
*      H5SC_chunk_t *lru_tail_ptr - The pointer to the tail of this dataset's chunk
*      LRU list (indicating it is the least recently used chunk). Set to Null
*      by default, when the chunk LRU list is empty, or when the chunk LRU
*      list contains a single element.
*
*      H5SC_dset_header_t *next_dset_ptr - The pointer to the next dataset in the
*      dataset LRU list. Set to Null by default, when this header is at the
*      tail of the dataset LRU list, or when the list has a single element.
*
*      H5SC_dset_header_t *prev_dset_ptr - The pointer to the previous dataset in
*      the dataset LRU list. Set to Null by default, when this header is at
*      the head of the dataset LRU list, or when the list has a single
*      element.
*
*      H5SC_dset_header_t *next_exhausted_ptr; - The pointer to the next dataset in
*      the exhausted dataset linked list. Set to Null by default, when this
*      dataset is at the exhausted list tail, or if the exhausted list has a
*      single element.
*
*      H5SC_dset_header_t *prev_exhausted_ptr; - The pointer to the previous
*      dataset in the exhausted dataset linked list. Set to Null by default,
*      when this dataset is at the exhausted list head, or if the exhausted
*      list has a single element.
*
*      UT_hash_handle dset_hval - A hash handle is required for this structure to
*      be hashable by UTHash; it is used to track this dataset in the SCC
*      dataset hash table. Set to Null by default.
*
*****/

struct H5SC_dset_header_t {
    uint64_t      magic;
    haddr_t       dset_addr;
    size_t        min_dset_size;
    size_t        chunk_lru_len;
    size_t        curr_dset_size;
    H5SC_tag_t    last_tag;
    H5SC_io_info_t *io_info;
    bool          resize_in_progress;
    bool          evict_exhausted;
    H5D_t         *dset;
    H5SC_chunk_t  *lru_head_ptr;

```

```

    H5SC_chunk_t      *lru_tail_ptr;
    H5SC_dset_header_t *next_dset_ptr;
    H5SC_dset_header_t *prev_dset_ptr;
    H5SC_dset_header_t *next_exhausted_ptr;
    H5SC_dset_header_t *prev_exhausted_ptr;
    UT_hash_handle     dset_hval;
} H5SC_dset_header_t;

```

Design Notes

This structure is created when a dataset is either created (through `H5Dcreate()`, with `H5D__create()` calling `H5SC_dset_create_header()`) or when a dataset is opened (`H5D__open_oid()` will call `H5SC_dset_create_header()`). This structure is also added to the dataset hash table and the dataset LRU list. There is no other valid way of creating an instance of `H5SC_dset_header_t`. When a dataset is closed (with `H5Dclose()`, leading to `H5D__close()` being called), the structure is removed from the dataset LRU list, removed from the dataset hash table, and the header is destroyed (with `H5SC_dset__destroy_header()`).

2.6. H5SC_stats_t – Per-File Global Cache Statistics

```

/*****
 *
 * Structure: H5SC_stats_t
 *
 * Purpose
 *     Maintains debugging and test statistics for a single SCC instance.
 *     These counters are used to observe cache behavior, particularly the
 *     rate at which chunk lookups resolve to resident non-placeholder
 *     chunks already present in the SCC.
 *
 * Fields
 *
 *     uint64_t scc_lookups
 *         The total number of SCC chunk lookup operations recorded for this
 *         cache instance.
 *
 *     uint64_t scc_hits
 *         The number of SCC chunk lookups that resolved to a resident
 *         non-placeholder chunk already present within the cache.
 *
 *     uint64_t scc_misses
 *         The number of SCC chunk lookups that did not resolve to a resident
 *         non-placeholder chunk already present within the cache.
 *
 *     uint64_t scc_inserts
 *         The number of chunk insert operations recorded for this cache
 *         instance.
 *
 *     uint64_t scc_removes
 *         The number of chunk remove operations recorded for this cache
 *         instance.
 *
 *     uint64_t scc_chunk_flush_count;
 */

```

```

*          The number of chunk flush operations recorded for this cache
*          instance.
*
*          uint64_t scc_dataset_flush_count;
*          The number of dataset flush operations recorded for this cache
*          instance.
*
*          uint64_t scc_evictions
*          The number of chunk or dataset eviction operations recorded for
*          this cache instance.
*
*          uint64_t scc_reads_batch_calls_count;
*          The number of individual read batches recorded for this cache
*          instance.
*
*          uint64_t scc_reads_last_batch_len;
*          The number of chunks processed in the last processed read batch
*          for this cache instance.
*
*          uint64_t scc_reads_max_batch_len;
*          The largest number of chunks processed in a single read batch
*          for this cache instance.
*
*          uint64_t scc_writes_batch_calls_count;
*          The number of individual write batches recorded for this cache
*          instance.
*
*          uint64_t scc_writes_last_batch_len;
*          The number of chunks processed in the last processed write batch
*          for this cache instance.
*
*          uint64_t scc_writes_max_batch_len;
*          The largest number of chunks processed in a single write batch
*          for this cache instance.
*
*****/

struct H5SC_stats_t {
    uint64_t scc_lookups;
    uint64_t scc_hits;
    uint64_t scc_misses;
    uint64_t scc_inserts;
    uint64_t scc_removes;
    uint64_t scc_reads_batch_calls_count;
    uint64_t scc_reads_last_batch_len;
    uint64_t scc_reads_max_batch_len;
    uint64_t scc_writes_batch_calls_count;
    uint64_t scc_writes_last_batch_len;
    uint64_t scc_writes_max_batch_len;
    uint64_t scc_chunk_flush_count;
    uint64_t scc_dset_flush_count;
    uint64_t scc_evictions;
};

```

Design Notes

This section is reserved and will be updated as we proceed with implementation.

2.7. H5SC__cache_config_t – Shared Chunk Cache Configuration Dataset-Based

```

/*****
 *
 * struct H5SC__cache_config_t
 *
 * The H5SC__scc_config_t structure is used to aggregate shared chunk cache
 * configuration data. The fields in the structure are
 * discussed individually below.
 *
 * version:      int field containing the version number of this version
 *                of the H5SC__scc_config_t structure. Any instance of
 *                H5SC__scc_config_t passed to the cache must have a known
 *                version number, or an error will be flagged.
 *
 * max_q_size:   size_t field containing the maximum number of bytes that
 *                the shared chunk cache may use while quiescent -- that is
 *                when not actively engaged in serving an I/O request
 *
 * max_a_size:   size_t field containing the maximum number of bytes that
 *                the shared chunk cache may use while serving an I/O request.
 *
 * ==== add other fields as required ====
 *
 *****/

typedef struct H5SC__cache_config_t {

    int version;

    size_t max_q_size;

    size_t max_a_size;

} H5SC__cache_config_t;

```

Design Notes

This structure may be expanded in future updates to accommodate additional user-specified configuration data for the Shared Chunk Cache.

2.8. H5SC_t – Per-File Global Cache Management

```

/*****
 *
 * Structure: H5SC_t
 *
 * Info
 *
 * The primary structure used for shared chunk cache management. Provides
 * fields for maintaining access to the head and tail of the dataset LRU
 * list, tracking the active and quiescent cache memory usage, and the
 * pointers necessary to access the hash tables used for chunk and
 * dataset lookups.
 *
 *****/

```



```

*
* Fields
*
*   size_t SCC_quiescent_size - The amount of quiescent data, in bytes, present
*   within the cache when there are no I/O operation being performed on
*   cached data. This field is computed using the sum of the
*   curr_dset_size field from each dataset currently in the SCC. This
*   field is updated whenever a chunk is added, removed, or resized while
*   present within the SCC. A value of 0 indicates the cache is empty. A
*   value of 0 is set by default when this structure is created.
*
*   size_t SCC_quiescent_limit - The amount of quiescent data, in bytes, that
*   should be retained within the cache when there is no I/O operation
*   being performed on cached data. This field is defined to be 1 GB by
*   default and may be set using a FAPL configuration (using the
*   max_q_size field in a H5SC__cache_config_t structure.)
*
*   size_t SCC_active_size - The amount of active data, in bytes, allocated to
*   the SCC when an I/O request is being processed. This field is an
*   integer-multiple of the current SCC_quiescent_size. The multiplier is
*   user configurable and is set to be 2x the size of the quiescent limit
*   by default to accommodate the application of filters or other
*   operations that often cause chunks to exceed estimated sizes. Once an
*   I/O request is completed, this field is set to 0.
*   --Currently not properly utilized; SCC_quiescent_size became the
*   catch-all for in-process size tracking. This
*   in a small refactor and is reserved as an optimization at this time.
*
*   size_t SCC_active_limit - The amount of active data, in bytes, that can be
*   processed at one time by the SCC. This quantity is treated as the
*   upper limit for available memory during I/O requests. It is set to be
*   2 GB default and may be set using a FAPL configuration (using the
*   max_a_size field in a H5SC__cache_config_t structure.)
*
*   size_t SCC_max_bytes - The hard cap, in bytes, for resident cache size.
*   When SCC_quiescent_size + new_allocation would exceed SCC_max_bytes,
*   eviction is required (or the request must be split by the caller).
*
*   size_t dset_lru_len - The number of datasets present within the SCC. This
*   value is used as the running count needed for DLL debugging and
*   testing.
*
*   H5SC_stats_t stats - Per-cache statistics used for SCC debugging and
*   testing. Tracks cache lookups, hits, misses, inserts, removes,
*   flushes, and evictions. These values are reset when the SCC is
*   created, destroyed, or reinitialized.
*
*   H5SC_dset_header_t *dset_lru_head_ptr - The pointer to the head of the
*   dataset LRU list (indicating it is the most recently used dataset).
*   Set to be Null by default or when the dataset LRU list is empty.
*
*   H5SC_dset_header_t *dset_lru_tail_ptr - The pointer to the tail of the
*   dataset LRU list (indicating it is the least recently used dataset).
*   Set to be Null by default or when the dataset LRU list is empty.
*
*   H5SC_chunk_t *chunk_hash_table_head_ptr - The pointer to the head of the SCC
*   chunk hash table. Set to Null when the hash table is empty or by

```

```

*          default. This pointer will only change when the first element of this
*          hash table is added or removed.
*
*          H5SC_dset_header_t *dset_hash_table_head_ptr - The pointer to the head of
*          the SCC dataset hash table. Set to Null when this hash table is empty
*          or by default. This pointer will only change when the first element of
*          this hash table is added or removed.
*
*****/

struct H5SC_t {
    size_t          SCC_quiescent_size;
    size_t          SCC_quiescent_limit;
    size_t          SCC_active_size;
    size_t          SCC_active_limit;
    size_t          dset_lru_len;
    H5SC_stats_t    stats;
    H5SC_dset_header_t *dset_lru_head_ptr;
    H5SC_dset_header_t *dset_lru_tail_ptr;
    H5SC_chunk_t     *chunk_hash_table_head_ptr;
    H5SC_dset_header_t *dset_hash_table_head_ptr;
} H5SC_t;

```

Design Notes

This structure is created when the file object is created (`H5Fcreate()`/`H5Fopen()` calls `H5F__new()`), which calls `H5SC_create()`. At the time of creation, a configuration structure may be passed to `H5Fcreate()` to set `SCC_quiescent_limit` and `SCC_active_limit`. When the file is closed (typically with `H5Fclose()`), this structure is destroyed with `H5SC_destroy()`. The `H5SC_t` associated with a file is stored in the shared file info.

2.9. H5SC_exhausted_list_t – List for Eviction-Related Dataset Exhaustion

```

/*****
*
* Structure: H5SC_exhausted_list_t
*
* Info
*
* The structure maintains a temporary doubly-linked list of dataset headers
* that have been fully processed during a single eviction pass within the
* Shared Chunk Cache (SCC). A dataset is added to this list once it can no
* longer contribute additional eviction candidates under the current policy
* (i.e., all eligible clean and dirty chunks have been considered or the
* dataset has reached its minimum reserved size).
*
* This list is used to prevent repeated scanning of the same dataset within
* a single eviction pass. After the eviction pass completes, all dataset
* headers stored in this list are restored to the global dataset LRU in a
* manner that preserves their relative ordering.
*
* Fields
*
* H5SC_dset_header_t *head

```

```

*   Pointer to the most recently added dataset header (MRU) in the exhausted
*   list. This corresponds to the head of the doubly-linked list.
*
*   H5SC_dset_header_t *tail
*   Pointer to the least recently added dataset header (LRU) in the exhausted
*   list. This corresponds to the tail of the doubly-linked list.
*
*   size_t len
*   Number of dataset headers currently stored in the exhausted list. This
*   field is maintained explicitly to support sanity checks and fast emptiness
*   evaluation.
*
*   size_t bytes
*   Aggregate of curr_dset_size for all dataset headers currently stored in
*   the exhausted list. This field is maintained as a sidecar accounting
*   mechanism and may be used for debugging, diagnostics, or future policy
*   extensions.
*
*****/

struct H5SC_exhausted_list_t {
    H5SC_dset_header_t *head;
    H5SC_dset_header_t *tail;
    size_t len;
    size_t bytes;
};

```

3 Theory of Operations

This section provides a conceptual overview of the five primary scenarios involving the cache: opening a file, writing to a file, reading from a file, flushing the cache, and closing a file. Where appropriate, dataset-specific interactions will also be discussed.

3.1. File Open

At the time an HDF5 file is opened, `H5SC_create()` will be called to perform the following actions:

- Allocate and initialize the file specific instance of `H5SC_t` with default configuration.
 - This is done using the `H5SC_create` function, which is called from `H5F__new()` as a part of the `H5Fcreate()` call stack.
- Examine the FAPL for shared chunk cache configuration. If found, overwrite configuration fields in the instance of `H5SC_t` as specified.
 - No current FAPL configuration; this will be implemented after the V1 is completed.

3.2. Dataset Open

The shared chunk cache must be informed whenever a relevant¹ dataset is either opened or created. On receipt of this notification, the shared chunk cache must:

¹ Datasets are supported if their I/O is managed by the shared chunk cache. Initially this means only sparse datasets. However, as the shared chunk cache is integrated into the HDF5 library, all dataset types will become relevant.

- Allocate and initialize an instance of `H5SC_dset_header_t`.
 - This is done within `H5D__create()`, which is called as a part of the public `H5Dcreate2()` call stack. The DAPL is available within `H5D__create()`.
- Scan the dataset access property list (DAPL) for non-default dataset configuration data, and if found, copy it into the instance of `H5SC_dset_header_t`.
- Insert the new instance of `H5SC_dset_header_t` into the shared chunk cache dataset hash table and dataset LRU list.

3.3. Dataset Flush

When directed, the shared chunk cache must:

- Find the target instance of `H5SC_dset_header_t` in the dataset hash table in the file specific instance of `H5SC_t`.
- Scan the chunk LRU list associated with the target instance of `H5SC_dset_header_t`.
- For each instance of `H5SC_chunk_t` on this list, determine if it is dirty, and flush it to file if so. Mark the instance of `H5SC_chunk_t` as clean when done. Note that dirty chunks are looked up on disk and if not found, are inserted into the on-disk chunk index as needed.
- If directed, evict all instances of `H5SC_chunk_t` associated with the dataset.

3.4. Dataset Erase

When performing an erase operation through the SCC, `H5Derase()` is used to remove or invalidate a selected region of a dataset without modifying the remaining in-bounds elements of the affected chunks. When a dataset uses structured chunk storage, the following actions will be taken with respect to the dataset's chunks:

- Convert the erase request into a list of erase operations on individual chunks and identify any chunks currently resident in the shared chunk cache.
- For each affected chunk:
 - Determine whether the erase selection fully covers the chunk or only partially overlaps it.
 - If the erase selection fully covers the chunk and the resulting chunk state no longer contains defined values, remove the chunk from the SCC chunk hash table and dataset LRU, delete the chunk from storage, and evict it from cache.
 - If the erase selection only partially overlaps the chunk, preserve all non-erased elements while removing the selected region according to the dataset erase and fill-value semantics.
- If an affected chunk is already resident in cache:
 - Apply the erase operation directly to the cached chunk image.
 - Update the chunk's dirty state and move the corresponding instance of `H5SC_chunk_t` to the head of its per-dataset LRU.

- If an affected chunk is not resident in cache but exists on disk:
 - Read the chunk's on-disk image.
 - Decode the chunk and apply any configured filters to reconstruct the in-memory representation.
 - Apply the erase operation to the selected region.
 - Re-encode and rewrite the chunk if the chunk still contains defined values, otherwise delete the chunk from storage.
- If an erase request targets a chunk or region that has never been materialized:
 - Avoid unnecessary chunk creation or materialization when the erase operation would not change the logical dataset contents.
 - Preserve sparse-storage behavior by skipping undefined regions whenever possible.
- Throughout erase processing, maintain SCC cache invariants by updating per-dataset LRUs, chunk hash-table membership, resident chunk accounting, dirty state tracking, and cached chunk metadata to reflect the resulting chunk state.

3.5. Applying Filters Through the SCC

When using filters with datasets routed through the SCC, filter processing is integrated directly into the structured chunk I/O path. Filters are configured on the dataset creation property list associated with a structured chunk dataset and participate transparently in normal SCC chunk lifecycle operations, including chunk materialization, cache residency management, eviction, erase processing, extent transitions, and reopen persistence.

When operating on a filtered structured chunk dataset, the following actions are performed with respect to the dataset's chunks:

- Configure structured chunk storage through `H5Pset_layout()` and `H5Pset_struct_chunk()`.
- Install one or more filters through `H5Pset_filter2()` for the desired structured chunk section type.
- During chunk materialization or load:
 - Locate the target chunk through the SCC lookup and structured chunk indexing path.
 - Read the chunk's encoded on-disk image if the chunk is not already resident in cache.
 - Decode the chunk image and reconstruct the in-memory chunk representation through the configured filter pipeline.
 - Insert or promote the corresponding instance of `H5SC_chunk_t` within the SCC residency structures.
- During chunk flush or writeback:

- Construct the chunk's serialized on-disk representation.
- Apply the configured filter pipeline to the chunk buffer.
- Write the encoded chunk image to storage.
- Update SCC accounting and chunk residency state to reflect the resulting chunk state.
- If a filtered chunk is already resident in cache:
 - Operate directly on the decoded in-memory chunk representation.
 - Avoid unnecessary re-materialization or decode work while the chunk remains resident.
- If a filtered chunk must be evicted:
 - Flush and encode dirty chunk state as necessary.
 - Remove the corresponding chunk from the active SCC residency structures.
 - Preserve consistency between chunk hash membership, dataset LRUs, and SCC accounting state.
- Throughout filtered SCC operation:
 - Filtered datasets participate in the same batching, cache-accounting, and eviction workflows as unfiltered datasets.
 - Filtered chunks participate in erase processing, extent shrink/regrow handling, sparse chunk materialization, reopen persistence, and multidimensional chunk indexing workflows.
 - Temporary transient memory may be required during decode and re-encode operations while filtered chunks are resident in cache.
- Multiple filters may be applied to the same structured chunk dataset, and different structured chunk section types may participate in different filter pipelines depending on dataset configuration.

3.6. Dataset Change Extent

- When performing a dataset extent change operation through the SCC, the upper layers of the library will first update the dataset extent information, then invoke `H5SC_set_extent_notify()`. When a dataset uses structured chunk storage, the following actions will be taken with respect to the dataset's chunks:
- Locate the target instance of `H5SC_dset_header_t` within the file-specific instance of `H5SC_t` and iterate over all resident chunks in the dataset LRU.
- For each resident chunk:
 - Determine whether the chunk is now fully outside the updated dataset extent, partially outside the extent, or still fully valid.

- If the chunk is fully outside the extent, remove it from the SCC chunk hash table and dataset LRU, delete the chunk from storage, and evict it from cache.
- If the chunk partially overlaps the new extent boundary, erase or fill the newly invalid region according to the dataset fill-value and erase semantics while preserving all still-valid in-bounds elements.
- After resident pruning is complete, recompute the logical chunk index for all remaining resident chunks using the updated dataset dimensions. Remove each chunk from the SCC chunk hash table, update its logical chunk coordinate and associated hash-table key, then reinsert it into the hash table under the new index.
- Perform a second pruning pass over chunks that may exist only in storage and are not currently resident in cache:
 - Identify chunks intersecting the region removed from the dataset extent.
 - If a chunk lies completely outside the new extent, delete it directly from storage.
 - If a chunk partially overlaps the new extent boundary and exists on disk, read and decode the chunk, erase or fill the newly invalid region, then re-encode and write the chunk back to storage.
- If the dataset extent grows and early allocation or fill-value policy requires physical chunk creation:
 - Allocate newly introduced chunks.
 - Initialize the chunk contents according to the dataset fill-value settings.
 - Apply any configured filters and write the initialized chunks to storage.
- Throughout extent processing, maintain SCC cache invariants by updating per-dataset LRUs, chunk hash-table membership, resident chunk accounting, and cached chunk metadata to reflect the updated dataset dimensions and chunk validity state.

3.7. Dataset Close

The shared chunk cache must be informed whenever a dataset is closed. On receipt of this notification, the shared chunk cache must:

- Find the target instance of `H5SC_dset_header_t` in the dataset hash table in the file specific instance of `H5SC_t`.
 - Uses `H5SC_ht_dset_find()`
- Verify that the chunk LRU associated with the target instance of `H5SC_dset_header_t` is empty. Otherwise, any remaining datasets should be flushed and then have any remaining `H5SC_chunk_t` elements evicted.
- Unlink the target instance of `H5SC_dset_header_t` from the dataset hash table and dataset LRU list.
- Free the target instance of `H5SC_dset_header_t`.

3.8. Writing Data Through the SCC

When writing data through the SCC from an in-memory data buffer to an HDF5 file, `H5SC_write()` is used². When a dataset uses structured chunk storage, the following actions will be taken with respect to the dataset's chunks:

- Convert the write request into a list of write requests on individual chunks. Identify any chunks that are currently resident in the shared chunk cache³.
- Determine the amount of shared chunk cache space that can be made available for this I/O request. This will be a combination of existing free space and evict-able chunks in the shared cache. Call this value *S*.
- Estimate the total size of the chunks in this I/O request in bytes (for resident chunks, use their cached size; for non-residents, use the size of the chunk in bytes as reported by the `H5O_layout_struct_chunk_t` structure⁴). If necessary, process a subset of the chunks whose aggregated size does not exceed *S*.
- For each such sub-list:
 - Evict sufficient entries from the shared chunk cache to make space for the target chunks in the sub-list. When flushing dirty entries, each chunk is processed individually⁵, including construction of its on-disk image and any necessary chunk index updates.
 - Create or load the target chunks. When loading the on-disk images of existing chunks, read the target on-disk images individually, and then decode and apply the appropriate filter(s) to the on disk buffers.
 - Apply the write requests from the sub-list to the now in cache chunks, moving each instance of `H5SC_chunk_t` touched to the head of its per dataset LRU, and its host instance of `H5SC_dataset_header_t` to the head of the dataset LRU.

3.9. Reading Data Through the SCC

When reading raw data through the SCC, `H5SC_read()` is used. Similar to the write process, if datasets in the I/O request are supported by the SCC, they are processed in the following way⁶:

² At the time of writing, `H5SC_write` is used to write directly to file in the Version 0 implementation of the cache, and does not convert the chunk buffer into the cache memory as outlined in this section. As a fully functional version of the SCC is implemented, this will be adjusted accordingly.

³ In a future update, resident chunks may be processed immediately by applying writes directly to their in-cache representations, followed by the necessary LRU updates.

⁴ This value is used as a conservative upper bound for planning purposes and may be refined with more accurate per-chunk sizing in future implementations.

⁵In the future, this could be done with a vector of chunks.

- Convert the read request into a list of read requests on individual chunks. Identify any chunks that are currently resident in the shared chunk cache⁷.
- Determine the amount of space that can be made available for this I/O request. This will be a combination of existing free space and evict-able chunks in the shared cache. Call this value S.
- Estimate the total size of the chunks in this I/O request in bytes (for resident chunks, use their cached size; for non-residents, use an estimated size based on layout information). If necessary, partition the request into one or more chunk I/O request sub-lists such that the aggregated size of each sub-list does not exceed S.
- For each such sub-list:
 - Evict sufficient entries from the shared chunk cache to make space for the target chunks in the sub-list. When flushing entries, each chunk is processed individually, including construction of its on-disk image and any necessary chunk index updates.
 - Load the target chunks from file. When loading existing chunks, read the on-disk images individually, then decode and apply the appropriate filter(s) to produce the in-memory chunk representation⁸.
 - Apply the read requests from the sub-list to the now in cache chunks, moving each instance of `H5SC_chunk_t` touched to the head of its per dataset LRU, and its host instance of `H5SC_dataset_header_t` to the head of the dataset LRU.

3.10. Flush the Cache

Should all data present in the SCC need to be flushed, `H5SC_flush()` will be utilized. This function will flush all dirty data present in the cache and may also be used to evict members of a dataset. This process will be user-configurable, allowing for per-dataset control of how a cache-wide flush operation should function. This operation is done on a per-dataset basis using the procedure for flushing datasets described above.

3.11. File Close

When an HDF5 file is closed, `H5SC_destroy()` is used to destroy the current SCC. This process does not flush chunks, and frees all data within the SCC using the following procedure:

- Verify that the shared chunk cache is empty.
- Free the file's instance of `H5SC_t`.

⁶ Similarly to the Write to Cache section, read operations utilize the structured chunk callbacks from `H5Dstruct_chunk.c`. The detailed behavior of these callbacks is omitted from this section.

⁷ In a future update, resident chunks may be processed immediately by applying writes directly to their in-cache representations, followed by the necessary LRU updates.

⁸ If a chunk in a read I/O request is not present on-disk, the chunk returned to the user will contain all fill values.

4 Detailed Component Discussions

This section provides descriptions for the various components or operations, such as the hash table and eviction candidate selection.

4.1. Chunk Tracking Through the Hash Table

To maintain consistent, constant-time lookups for chunks, a globally available hash table will be utilized through the use of UTHash. The global hash table utilized by the SCC will rely on the mechanisms provided by UTHash, through the inclusion of a single C header file. Continuing from existing HDF5 paradigms, UTHash utilizes the Jenkins hash function by default, while providing access to additional options should alternate be desirable. For each hash table item, the key, payload (i.e., the values), and the hash handle may be maintained in a single structure. The proposed `H5SC_chunk_key_t` is well suited to serve as a key when using UTHash, and provides a unique, data-focused identification for each individual chunk⁹. The macros for UTHash are well documented [3] and could easily be modified if need be.

4.2. Eviction

Eviction in the Shared Chunk Cache (SCC) is performed using a hierarchical LRU policy with strict per-dataset draining semantics. The global dataset LRU identifies the least-recently-used dataset as the starting point for eviction. Once selected, eviction proceeds within that dataset by traversing its chunk-level LRU from tail to head. All eligible clean chunks are evicted first, followed by dirty chunks (which are flushed and then evicted) if additional space reclamation is required. Eviction within a dataset continues until either sufficient space has been freed to satisfy the request or the dataset reaches its reserved minimum size (`min_dset_size`). At that point, the dataset is considered fully processed for the current eviction pass and is temporarily removed from the global dataset LRU and placed into an auxiliary exhausted-dataset list to avoid redundant rescanning. The eviction process then advances to the next dataset in global LRU order and repeats. After the eviction pass completes, all datasets in the exhausted list are restored to the global dataset LRU in their original relative order. This approach ensures deterministic, dataset-local eviction behavior while preventing repeated traversal of datasets that can no longer contribute eviction candidates within the current pass.

The `partial_io` flag provides identification of chunks that were only partially read or written¹⁰; such chunks will be given a second pass through the cache in favor of fully read or written chunks. Eviction continues within a dataset until its reserved minimum (`min_dset_size`) has been reached, all chunks have been processed, or the required amount of space within the SCC has been freed.

⁹ As a general note, UTHash does not automatically do any verification concerning the uniqueness of a given key. However, the `HASH_FIND` macro is provided by UTHash to allow the user to check whether a key is present within the hash table.

¹⁰ A chunk is considered partially read or written when the selection describing the chunk intersecting the selection on the dataset results in the selected chunk not matching the full chunk selection. This may also occur when the chunk dimension(s) don't evenly divide the dataset dimension(s). Note that a partial/edge chunk can be considered fully read or written if all elements in that chunk are processed in the I/O request.

After the `min_dset_size` has been reached with a dataset, it is temporarily removed from the dataset LRU list and added to a list of datasets which have exhausted their ability to nominate further eviction candidates¹¹. **-Not currently implemented.**

4.2.1 Additional Eviction Strategies

The current design supports extension beyond the default two-stage LRU approach. Alternative strategies such as Least Frequently Used (LFU) or Most Recently Used (MRU) can be implemented with minimal changes:

- Frequency-based policies can use the integer counter in each chunk (`chunk_counter`).
- Order-based policies can reuse the doubly linked lists already maintained for datasets and chunks.
- Dataset-level configuration may eventually expose options to select or tune eviction policies.

This flexibility allows new policies to be added without significant restructuring of the SCC.

4.3. Batching

Batching in the H5SC module is implemented as a dataset-scoped, windowed execution model that separates I/O scheduling from per-chunk processing. For each dataset in a request, `H5SC__io_info_init()` constructs a dataset-owned vector of selected chunk descriptors (`sel_chunks`) that persists for the duration of the operation. A central dispatcher (the `H5SC_invoke_*` and `H5SC__invoke_*_dset_batched` routines) drives execution by processing this array in successive subranges defined by (`sel_start`, `num_sel_chunks`). Each subrange represents a batch of chunks that can be processed within current cache constraints.

Batch sizing is determined by SCC memory policy. The dispatcher evaluates the incremental memory cost of bringing chunks into the cache, along with available free space (including reclaimable clean and, if necessary, dirty data), to ensure that processing a batch does not exceed `SCC_active_limit`.

Within each batch, lookup metadata is prepared once and reused to identify cache hits and misses and to avoid redundant layout queries. The worker functions (`H5SC_read()` and `H5SC_write()`) then operate only on the active batch, performing per-chunk operations such as index resolution, data movement (gather/scatter), encoding/decoding, and read or write application.

This separation allows the invoke layer to manage scheduling, batching, and cache space decisions, while the worker functions remain focused on deterministic per-chunk processing. As a result, the design provides predictable memory usage, reduces repeated lookup overhead, and integrates cleanly with SCC eviction and accounting mechanisms.

¹¹ A dataset is considered exhausted when, during eviction, the `min_dset_size` has been achieved, signifying that no additional `H5SC_chunk_t` elements may be removed from this dataset. To avoid potentially repeated processing of this now exhausted dataset, it is temporarily removed from the dataset LRU list and added to a separate linked list. After the eviction process is completed datasets on the exhausted list are returned to the dataset LRU list.

4.4. Public Configuration Structure

The structure will be modeled after the `H5AC_cache_config_t` structure available in `H5ACpublic.h`. The goal of this publicly available structure is to enable users to set dataset-specific minimum retention values, eviction strategies, and global memory limits (active, quiescent, and maximum available). Default values will be included as well, serving as baseline values.

4.5. Filter Configuration and Usage

Filters for SCC-managed structured chunk datasets are configured through the dataset creation property list associated with the dataset. Structured chunk filtering uses the standard HDF5 filter pipeline while extending filter installation to operate on structured chunk section types.

When configuring a filtered structured chunk dataset, the following steps are performed:

- Create a dataset creation property list configured for structured chunk storage:
 - Configure the dataset layout through `H5Pset_layout()`.
 - Configure structured chunk storage through `H5Pset_struct_chunk()`.
- Install one or more filters through `H5Pset_filter2()` for the desired structured chunk section type:
 - `H5_SECTION_FIXED` for fixed structured chunk payload regions.
 - `H5_SECTION_SELECTION` for selection-managed structured chunk metadata regions.
- Configure the filter-specific parameters required by the selected filter pipeline.
- Create the dataset using the configured dataset creation property list.

Structured chunk filtering integrates directly with the SCC read/write path. During chunk write-back, SCC serializes the chunk's in-memory representation, applies the configured filter pipeline, and writes the encoded image to storage. During chunk load or materialization, SCC reads the encoded chunk image, reverses the configured filter pipeline, and reconstructs the in-memory chunk representation prior to servicing the I/O request.

Filtered SCC datasets participate in the same operational workflows as unfiltered datasets, including:

- chunk materialization,
- chunk eviction,
- batching,
- erase operations,
- extent shrink/regrow processing,

- reopen persistence,
- sparse chunk materialization,
- multidimensional chunk indexing,
- and mixed filtered/unfiltered dataset operation within the same file.

4.5.1 GZIP / DEFLATE Filtering

GZIP filtering is configured through:

- `H5Z_FILTER_DEFLATE`
- and a compression-level parameter supplied through `H5Pset_filter2()`.

GZIP filtering has been validated within SCC-managed structured chunk datasets across:

- `H5S_ALL` I/O,
- explicit hyperslab selections,
- sparse point-like access patterns represented through hyperslab selections,
- multidimensional datasets,
- extensible datasets,
- erase/rewrite workflows,
- extent shrink/regrow operations,
- reopen persistence,
- forced eviction and batching scenarios,
- and mixed-filter pipeline configurations.

Common GZIP-based filter pipelines include:

- GZIP only,
- SHUFFLE + GZIP,
- FLETCHER32 + GZIP,
- and SHUFFLE + FLETCHER32 + GZIP.

4.5.2 SZIP Filtering

SZIP filtering is configured through:

- `H5Z_FILTER_SZIP`

- and the coding-mode and pixels-per-block parameters supplied through `H5Pset_filter2()`.

Current SCC validation has exercised:

- nearest-neighbor (`NN`) coding mode,
- entropy-coding (`EC`) mode,
- and multiple pixels-per-block configurations.

SZIP filtering has been validated within SCC-managed structured chunk datasets across:

- full-dataset I/O,
- multi-chunk hyperslab I/O,
- partial-chunk I/O,
- sparse point-like access patterns represented through hyperslab selections,
- multidimensional datasets,
- extensible datasets,
- erase operations,
- extent shrink/regrow workflows,
- reopen persistence,
- forced eviction and batching scenarios,
- mixed-rank filtered dataset stress workloads,
- and mixed filter-pipeline configurations.

Current SZIP support applies to:

- `H5_SECTION_FIXED` structured chunk regions.

Selection-managed structured chunk metadata regions are not currently considered valid SZIP targets.

4.5.3 Mixed Filter Pipelines

Structured chunk datasets may apply different filter pipelines to different structured chunk section types. This allows fixed payload regions and selection-managed metadata regions to participate in separate filter pipelines within the same dataset.

Validated mixed-filter configurations include:

- GZIP-filtered selection metadata with SZIP-filtered fixed payload regions,
- mixed filtered and unfiltered datasets within the same file,
- and simultaneous operation across datasets with different filter pipelines and dimensionality.

Filtered SCC datasets participate in the same SCC accounting, residency, batching, and eviction policies as unfiltered datasets. While resident, filtered chunks may temporarily require additional transient memory during decode and re-encode operations.

5 Top Level API

These functions are called by the upper layers of the HDF5 library and serve as initial entry points to the Shared Chunk Cache (H5SC) package. The contents of this section largely originated in [2] and have been updated/modified where appropriate.

```
H5SC_t *H5SC_create(H5F_t *file, H5P_genplist_t *fa_plist,
H5SC__cache_config_t *config_ptr);
```

Creates a new, empty shared chunk cache. Will be called at file open time.

```
herr_t H5SC_destroy(H5SC_t *cache);
```

Destroys a shared chunk cache, freeing all data used. Does not flush chunks. Called at file close time.

```
herr_t H5SC_invoke_write(H5SC_t *cache, size_t count, H5D_dset_io_info_t
*dset_info);
```

Writes raw data through the shared chunk cache. Called by `H5D__write()`; used to initiate internal generic setup and batching through the SCC. Replaced `H5SC_write()`, which is now functionally an internal routine.

There may be datasets in the `dset_info` vector that do not support the shared chunk cache. These datasets must be ignored by the shared chunk cache. here may also be datasets that have `skip_io` set. For datasets in `dset_info` that do support the shared chunk cache, the intermediate buffers will need to be freed after the data has been written to the file.

```
herr_t H5SC_invoke_read(H5SC_t *cache, size_t count, H5D_dset_io_info_t
*dset_info);
```

Reads raw data through the shared chunk cache. Called by `H5D__read()`; used to initiate initial generic setup and batching through the SCC. Replaced `H5SC_read()`, which is now functionally an internal routine.

There may be datasets in the `dset_info` vector that do not support the shared chunk cache. These datasets must be ignored by the shared chunk cache. There may also be datasets that have `skip_io` set. These datasets must also be skipped. For datasets in `dset_info` that do support the shared chunk cache, once the data that has been scattered into the user buffer, the intermediate buffers will need to be freed using the `H5SC_chunk_evict_t` callback.

```
herr_t H5SC_read(H5SC_t *cache, size_t count, H5D_dset_io_info_t
*dset_info);
```

Reads raw data through the shared chunk cache. Called by `H5SC_invoke_read_dset_batched()` after initial generic setup.

```
herr_t H5SC_write(H5SC_t *cache, size_t count, H5D_dset_io_info_t
*dset_info);
```

Writes raw data through the shared chunk cache. Called by `H5SC__invoke_write_dset_batched()` after initial generic setup.

```
herr_t H5SC_flush(H5SC_t *cache);
```

Flushes all cached data.

```
herr_t H5SC_flush_dset(H5SC_t *cache, H5D_t *dset, bool evict);
```

Flushes all data cached for a single dataset. If `evict` is true, also evicts all cached data.

```
herr_t H5SC_direct_chunk_read(H5SC_t *cache, H5D_t *dset, hsize_t *offset,
void *udata, void *buf, size_t *buf_size);
```

Reads the chunk that starts at coordinates give by offset directly from disk to `buf`, without any decoding or conversion. First flushes that chunk if it is dirty in the cache. If `buf_size` is not large enough to read the entire chunk, nothing is read, and the needed size is returned in `*buf_size`. Not currently implemented. **-Not currently implemented.**

```
herr_t H5SC_direct_chunk_write(H5SC_t *cache, H5D_t *dset, hsize_t *offset,
void *udata, const void *buf);
```


Writes the chunk that starts at coordinates given by offset directly from `buf` to disk, without any encoding or conversion. First invalidates and evicts that chunk from cache if it is present. Not currently implemented. **-Not currently implemented.**

```
H5S_t *H5SC_get_defined(H5SC_t *cache, H5D_t *dset, const H5S_t
*file_space);
```

Returns a copy of `file_space` with only elements selected that are both selected in `file_space` and defined in `dset`. If `file_space` uses a point selection, the ordering of selected points will be preserved in the returned data space. Currently returns a copy of the `file_space` (partial functionality).

```
herr_t H5SC_erase(H5SC_t *cache, H5D_t *dset, const H5S_t *file_space);
```

Causes the elements selected in `file_space` to become undefined in `dset`. If `dset` does not support tracking defined elements, returns an error. Currently being implemented.

```
herr_t H5SC_set_extent_notify(H5SC_t *cache, H5D_t *dset, const hsize_t
*old_dims);
```

Called after `H5Dset_extent()` has been called for a dataset, so the cache can recompute chunk indices, delete chunks, clear unused sections of chunks, etc. Currently being implemented.

```
herr_t H5SC_validate_config(const H5SC__cache_config_t *config_ptr);
```

Runs a sanity check on the contents of the supplied instance of `H5SC__cache_config_t`. Related to the integration of File Access Property List (FAPL) support.

```
H5SC_dset_header_t *H5SC_dset_create_header(H5SC_t *cache, haddr_t addr,
size_t max_chunk_size);
```

Allocates and initializes a `H5SC_dset_header_t` structure with the specified dataset address. Does not add this structure to the hash table or appropriate LRU list. Necessary for `H5D__create` and `H5D__open_oid()` to operate properly. If the default minimum dataset size (10 MB) is greater than the quiescent limit for the SCC, the minimum dataset size is set to be two times the max chunk size. This necessary to prevent odd eviction behavior when the quiescent limit is small (mostly useful for testing).

```
herr_t H5SC_dset_destroy_header(H5SC_t *cache, H5D_t *dset,
H5SC_dset_header_t *dset_hdr);
```

Frees any remaining chunks, persistent io_info, and frees the dataset header structure. Assumes that the dataset has been freed from the dataset LRU and dataset hash table.

```
herr_t H5SC_dset_is_empty(H5SC_dset_header_t *dset_hdr, bool *is_empty);
```

Helper function that inspects a dataset header to ensure no further chunk data is present. Due to how `H5SC_dset_header_t` is defined in the H5SC module, this is necessary for instances where a different module must interact with a field in this structure (such as within `H5D_close()`).

```
haddr_t H5SC_dset_get_addr(H5SC_dset_header_t *dset_hdr);
```

Function used to get the address embedded within a dataset header structure. Due to how `H5SC_dset_header_t` is defined, this is necessary for instances where a different module must interact with a field in this structure (such as within `H5D_close()`).

```
herr_t H5Pset_scc_config(hid_t plist_id, H5SC__cache_config_t *config_ptr);
```

Function used to set the shared chunk cache configuration data in the provided File Access Property List (FAPL). Currently only supports setting non-zero values for the quiescent and active size limits.

```
herr_t H5Pget_scc_config(hid_t plist_id, H5SC__cache_config_t *config
/*out*/);
```

Function used to retrieve the current (or default) shared chunk cache configuration from the provided File Access Property List (FAPL). Note that this function will return an error if `config` is NULL or if `config_ptr->version` specifies an unknown version of `H5AC_cache_config_t`. Expect to use `H5SC__CURR_SCC_VERSION` when defining the `H5SC__cache_config_t` structure.

6 SCC Internal Functions

Functions that are internal to the SCC operations. Functions are presented in categories based on their primary functionality.

6.1 Hash table specific functions

```
H5SC_chunk_t *H5SC__ht_chunk_find(H5SC_t *cache, const H5SC_chunk_key_t
*chk_key);
```

Helper function used to lookup a chunk record by it's unique (per file) `H5SC_chunk_key_t` and returns the stored pointer.

```
herr_t H5SC__ht_chunk_insert(H5SC_t *cache, H5SC_chunk_t *chk);
```

Helper function that inserts a chunk record into the chunk hash table. A unique `H5SC_chunk_key_t` is used as the key (unique for each file).

```
herr_t H5SC__ht_chunk_delete(H5SC_t *cache, const H5SC_chunk_key_t
*chk_key);
```

Helper function that unlinks (removes) a chunk from the chunk hash table by address. This does not free the unlinked node; this is left to the caller.

```
H5SC_dset_header_t *H5SC__ht_dset_find(H5SC_t *cache, haddr_t addr);
```

Helper function used to lookup a dataset header record by its unique (per file) `haddr_t` and returns the stored pointer.

```
herr_t H5SC__ht_dset_insert(H5SC_t *cache, H5SC_dset_header_t *dset_hdr);
```

Helper function that inserts a dataset header record into the dataset hash table. The dataset object address header is used as the key (unique for each file).

```
herr_t H5SC__ht_dset_delete(H5SC_t *cache, haddr_t addr);
```

Helper function that unlinks (removes) a dataset header from the dataset header hash table by address. This does not free the unlinked node; this is left to the caller.

6.2. DLL specific functions

```
herr_t H5SC__dset_lru_prepend(H5SC_t *cache, struct H5SC_dset_header_t
*dset_hdr);
```

Helper function used to insert a dataset header from the dataset LRU list. Decrements the dataset chunk counter and SCC quiescent size.

```
herr_t H5SC__dset_lru_promote(H5SC_t *cache, H5SC_dset_header_t *dset_hdr);
```

Helper function used to move a dataset header to the head of the dataset LRU list.

```
herr_t H5SC__dset_lru_remove(H5SC_t *cache, struct H5SC_dset_header_t
*dset_hdr);
```

Helper function used to remove a dataset header from the dataset LRU list. Decrements the dataset chunk counter and SCC quiescent size.

```
static herr_t H5SC__dset_exhausted_lru_prepend(H5SC_exhausted_list_t *exh,
struct H5SC_dset_header_t *dset_hdr);
```

Helper function used to prepend a dataset to the head exhausted dataset list.

```
static herr_t H5SC__dset_exhausted_remove(H5SC_exhausted_list_t *exh,
struct H5SC_dset_header_t *dset_hdr);
```

Helper function used to remove a dataset from the exhausted dataset list.

```
static herr_t H5SC__dset_exhausted_restore_to_global_lru(H5SC_t *cache,
H5SC_exhausted_list_t *exh);
```

Helper function used to restore an exhausted dataset to the tail of the global dataset LRU list.

```
herr_t H5SC__chunk_lru_prepend(struct H5SC_dset_header_t *dset_hdr, struct
H5SC_chunk_t *chunk);
```

Helper function used to insert a chunk at the head of the appropriate dataset's LRU list. Increments the dataset chunk counter and dataset size. Does not interact with SCC quiescent size; this is left as a responsibility of the caller and may not always be desirable.

```
herr_t H5SC__chunk_lru_remove(struct H5SC_dset_header_t *dset_hdr, struct
H5SC_chunk_t *chunk);
```

Helper function used to remove a chunk from the appropriate dataset's LRU list. Decrements the dataset chunk counter and dataset size. Does not interact with SCC quiescent size; this is left as a responsibility of the caller and may not always be desirable.

6.3. H5SC_chunk_t specific functions

```
static herr_t H5SC__compute_logical_chunk_index(unsigned ndims, const
hsize_t *dset_dims, hsize_t *chunk_dims, const hsize_t *elem_coord, hsize_t
*log_chk_idx);
```

Helper function used to compute the logical chunk coordinate. This value is unique within a dataset, but not across individual datasets.

```
static inline void H5SC__set_interleave_bit(H5SC_chunk_key_t *chunk_key,
unsigned bit_position);
```

Helper function used to set an interwoven bit (0...127) in the 128-bit chunk key structure.

```
static herr_t H5SC__compute_chunk_key(haddr_t *dset_object_header_addr,
hsize_t *log_chk_coord, H5SC_chunk_key_t *chunk_key);
```

Helper function used to compute the chunk key associated with a dataset object header and chunk logical coordinate pair.

```
static H5SC__chunk_t *H5SC_make_chunk(H5SC_chunk_key_t key, size_t
cached_sz, size_t counter, bool pio);
```

Function that initializes an `H5SC_chunk_t` instance for a specific `H5SC_chunk_key_t`. This does not add the newly created chunk to the hash table or appropriate LRU list.

```
static herr_t H5SC__chunk_tearardown_resident(H5D_t *dset, H5SC_chunk_t
*chunk);
```

Helper function used to teardown a resident decoded chunk object held by the SCC. Utilizes the evict callback function (`H5D__struct_chunk_evict()`).

6.4. I/O specific functions

```
static inline void H5SC__io_sel_chunk_init(H5SC_io_sel_chunk_t *c);
```

Helper function used to initialize an instance of `H5SC_io_sel_chunk_t` with safe default values.

```
static herr_t H5SC__io_info_init(H5SC_t *cache, size_t count,
H5D_dset_io_info_t *dset_info);
```

Helper function used to initialize `sc_io_info` for an I/O operation, including computing the per-chunk memory and file data spaces. Additionally, this function allocates and fills in the `sc_io_info→sel_chunks` array, which is the array of chunks that are involved in this I/O operation. Chunks not yet in the cache that are present in this I/O operation have a placeholder created, which is treated as a non-resident chunk until it has been processed.

```
static herr_t H5SC__erase_io_info_init(H5SC_t *cache, H5D_t *dset, const
H5S_t *file_space);
```

Helper function derived from `H5SC__io_info_init()` that is called by `H5SC_erase()` to identify file space information necessary for the erase operation.

```
static herr_t H5SC__io_info_reset(H5SC_io_info_t *sc_io_info);
```

Helper function used to reset the contents of an `sc_io_info` structure to default values for later reuse.

```
static herr_t H5SC__io_info_term(H5SC_io_info_t *sc_io_info);
```

Helper function used to free all memory referenced by an `sc_io_info` structure. Does not free the `sc_io_info` structure itself.

```
static herr_t H5SC__invoke_read_dset_batched(H5SC_t *cache,
H5D_dset_io_info_t *dset_info, H5SC_dset_header_t *dset_hdr);
```

Helper function that is responsible for determining whether batching is necessary to complete the requested read operation. Currently offers two modes; a chunk-by-chunk mode that processes exactly one chunk per call to `H5SC_read()` and size-restricted batching mode. The chunk-by-chunk mode has primarily been used as a debugging tool during development. The size-restricted batching mode operates as a greedy algorithm and will attempt to process as many chunks as possible in each batch based on the available memory (SCC active limit) and the amount of space that can be freed, as computed by other helper functions. Batch sizes need not be uniform.

```
static herr_t H5SC__invoke_write_dset_batched(H5SC_t *cache,
H5D_dset_io_info_t *dset_info, H5SC_dset_header_t *dset_hdr);
```

Helper function that is responsible for determining whether batching is necessary to complete the requested write operation. Currently offers two modes; a chunk-by-chunk mode that processes exactly one chunk per call to `H5SC_write()` and size-restricted batching mode. The chunk-by-chunk mode has primarily been used as a debugging tool during development. The size-restricted batching mode operates as a greedy algorithm and will attempt to process as many chunks as possible in each batch based on the available memory (SCC active limit) and the amount of space that can be freed, as computed by other helper functions. Batch sizes need not be uniform.

```
static herr_t H5SC__lookup_cache_misses(H5D_t *dset, H5SC_io_info_t
*sc_io_info);
```

Helper function that uses the lookup callback (`H5D__struct_chunk_lookup()`) to lookup the on disk chunk addresses and chunk sizes that are not currently a resident of the SCC. Also utilized during batching.

6.5. Eviction specific functions

```
static herr_t H5SC__flush_one_chunk(H5SC_t *cache, H5D_t *dset,
H5SC_dset_header_t *dset_hdr, H5SC_chunk_t *chk);
```

Helper function used to write one dirty, resident non-placeholder chunk to file. This process includes doing a non-destructive encode operation, allocating/updating the on-disk indexing, and then writing the chunk to file with a block write.

```
static herr_t H5SC__evict_one_chunk(H5SC_t *cache, H5D_t *dset,
H5SC_dset_header_t *dset_hdr, H5SC_chunk_t *chk);
```

Helper function used while evicting a single chunk from the SCC. This includes tearing down the resident into a decoded state, unlink it from the chunk LRU, remove it from the chunk hash table, and free the `H5SC_chunk_t` structure.

```
static bool H5SC__select_evict_candidate_in_dset(H5SC_dset_header_t
*dset_hdr, H5SC_chunk_t **cand_chk, H5SC_evict_mode_t mode);
```

Helper function used to select the tailmost eligible eviction candidate from a single dataset. The `H5SC_evict_mode_t` enum is currently only configured for two selections: `H5SC_EVICT_CLEAN_ONLY` (0) and `H5SC_EVICT_DIRTY_ONLY` (1). This enum will allow for additional eviction modes and will eventually be configurable through the `FAPL/DAPL`.

```
static bool H5SC__select_evict_candidate(H5SC_t *cache, H5SC_dset_header_t
**cand_hdr, H5SC_chunk_t **cand_chk, H5SC_evict_mode_t mode);
```

Helper function used to nominate a chunk as an eviction candidate. The `H5SC_evict_mode_t` enum is currently only configured for two selections: `H5SC_EVICT_CLEAN_ONLY` (0) and `H5SC_EVICT_DIRTY_ONLY` (1). This enum will allow for additional eviction modes and will eventually be configurable through the `FAPL/DAPL`.

```
static size_t H5SC__calc_freeable_clean_bytes(const H5SC_t *cache);
```

Helper function used to calculate the maximum number of bytes that could be freed by flushing and/or evicting clean chunks across all datasets present within the SCC without dropping any dataset below the `min_dset_size` constraint. Clean chunks will be considered prior to considering dirty chunks under the current eviction configurations.

```
static size_t H5SC__calc_freeable_dirty_bytes(const H5SC_t *cache);
```

Helper function used to calculate the maximum number of bytes that could be freed by flushing and/or evicting dirty chunks across all datasets present within the SCC without dropping any dataset below the `min_dset_size` constraint. Dirty chunks are typically considered after all eligible clean chunks have been processed. This function returns an additional reclaim estimate and is not an alternative to computing clean `freeable bytes` (i.e., `clean_freeable + dirty_freeable` is the total number of bytes that can currently be freed with respect to `min_dset_size`).

```
static herr_t H5SC__verify_candidate_is_tailmost_eligible(const H5SC_t
*cache, const H5SC_dset_header_t *cand_hdr, const H5SC_chunk_t *cand_chk,
bool allow_dirty);
```

Helper function as a sanity check to verify that a nominated eviction candidate is currently at the tail of the appropriate dataset. Useful for debugging and sanity checks.

6.6. Size accounting functions

```
static herr_t H5SC__ensure_space(H5SC_t *cache, size_t bytes_needed);
```

Helper function used to ensure the SCC has enough space to process an I/O request and enforces the SCC active limit. Calls eviction/flush routines for eligible eviction candidates until the requested space is freed or no additional candidates can be selected from this dataset.

```
herr_t H5SC__chunk_update_cached_size(H5SC_dset_header_t *dset_hdr, struct
H5SC_chunk_t *chunk, size_t new_size);
```

Helper function used to update the chunk size stored within an instance of a resident non-placeholder chunk.

```
static size_t H5SC__calc_inc_for_chunk(const H5D_t *dset, const
H5SC_chunk_t *chk, size_t desired);
```

Helper function that computes the bytes necessary to materialize/resize a chunk during batching.

```
static herr_t H5SC__trim_to_quiescent_limit(H5SC_t *cache);
```

Helper function used to enforce the SCC quiescent limit through flushing/evicting non-placeholder resident chunks. This is performed after an I/O request is completed.

```
static herr_t H5SC__account_dset_size_change(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, size_t old_size, size_t new_size);
```

Helper function used to apply a change in dataset size to the SCC quiescent size.

```
static herr_t H5SC__account_chunk_link_change(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, size_t old_dset_size);
```

Helper function that is used to update the current in-cache dataset size after an appropriate chunk DLL operation (insert/remove/resize). Also responsible for calling `H5SC__account_dset_size_change()` to update the quiescent size.

6.7. Extensible dimension-related functions

```
static bool H5SC__chunk_outside_extent(const H5D_t *dset, const hsize_t
*chunk_dims, const hsize_t *scaled);
```

Helper function used to determine whether a chunk lies completely outside a dataset's current extent.

```
static bool H5SC__chunk_is_partial_bound(unsigned ndims, const hsize_t
*chunk_dims, const hsize_t *scaled, const hsize_t *dims);
```

Helper function that is used to determine whether a chunk is considered to have a partial boundary (otherwise called an edge chunk) with respect to dims.


```
static bool H5SC__chunk_needs_erase(const H5D_t *dset, const hsize_t
*chunk_dims, const hsize_t *scaled, const hsize_t *old_dims);
```

Helper function used to determine whether a surviving chunk overlaps a newly invalidated region after extent shrinkage (which would require erasing values).

```
static herr_t H5SC__build_chunk_erase_selection(unsigned ndims, const
hsize_t *chunk_dims, const hsize_t *scaled, const hsize_t *new_dims, H5S_t
**erase_space /*out*/);
```

Helper function used to build the chunk-local selection of values now outside the new extent. The returned data space is in logical chunk coordinates.

```
static herr_t H5SC__rekey_dset_chunks_after_extent_change(H5SC_t *cache,
H5D_t *dset, H5SC_dset_header_t *dset_hdr, const hsize_t *chunk_dims);
```

Helper function used to recompute logical chunk coordinates and chunk keys (H5SC_chunk_key_t) for all chunks in a dataset after an extent change that alters the dataset's chunk grid.

```
static herr_t H5SC__erase_chunk_invalid_extent_region(H5D_t *dset, const
hsize_t *chunk_dims, const hsize_t *old_dims, H5SC_chunk_t *chk, size_t
*nbytes, size_t *alloc_size, bool *delete_chunk);
```

Helper function used to erase invalid values from a resident structured chunk after a dataset extent shrink. This function also erases any newly invalidated on-disk values using the SCC callback functions.

```
static herr_t H5SC__for_each_scaled_coord_outside_extent(H5SC_t *cache,
H5D_t *dset, unsigned ndims, const hsize_t *old_nchunks, const hsize_t
*new_nchunks);
```

Helper function used to iterate over the scaled chunk coordinates that were present in the old chunk grid but are now outside the new chunk grid after an extent shrink.

```
static herr_t H5SC__prune_one_scaled_coord_outside_extent(H5SC_t *cache,
H5D_t *dset, const hsize_t *scaled);
```

Helper function used to delete a single structured chunk coordinate that lies outside the dataset's new extent.

```
static herr_t H5SC__lookup_chunk_for_prune(H5D_t *dset, const hsize_t
*scaled, haddr_t *addr, hsize_t *disk_nbytes, bool *exists);
```

Helper function used to query on-disk storage for a chunk during extent-prune cleanup.

6.8. Statistics-specific functions

```
static void H5SC__stats_reset(H5SC_t *cache);
```

Responsible for resetting the counters present within an instance of `H5SC_stats_t`. Used when an instance of `H5SC_t` is created or destroyed.

```
static void H5SC__stats_record_lookup(H5SC_t *cache, hbool_t hit);
```

Used to record each time a chunk is looked up within the SCC (i.e., using `H5SC__ht_chunk_find()`). This count is used as the denominator when computing hit/miss rates.

```
static void H5SC__stats_record_insert(H5SC_t *cache);
```

Used to count the number of times chunks are inserted into the chunk-focused hash table. This does not reflect when a chunk is linked/relinked within a DLL.

```
static void H5SC__stats_record_delete(H5SC_t *cache);
```

Increases the count of chunks that have been removed from the SCC in situations where a chunk removal is logically distinct from eviction. It is currently used when a chunk is removed from the hash table but may be adapted later for tracking changes in extent or other interesting types of removals.

```
static void H5SC__stats_record_chunk_flush(H5SC_t *cache);
```

Used to count the number of individual chunk flushes within the SCC.

```
static void H5SC__stats_record_dset_flush(H5SC_t *cache);
```

Used to count the number of individual dset flushes within the SCC.

```
static void H5SC__stats_record_batch_read(H5SC_t *cache);
```

Used to count the number of individual read batches that have been processed within the SCC.

```
static void H5SC__stats_record_batch_write(H5SC_t *cache);
```

Used to count the number of individual write batches that have been processed within the SCC.

```
static void H5SC__stats_record_batch_read_len(H5SC_t *cache, size_t batch_len);
```

Used to record the size of the most recent read batch and update the maximum observed batch length (if applicable).

```
static void H5SC__stats_record_batch_write_len(H5SC_t *cache, size_t
batch_len);
```

Used to record the size of the most recent write batch and update the maximum observed batch length (if applicable).

```
static void H5SC__stats_record_eviction(H5SC_t *cache);
```

Used to count the times chunk eviction is triggered within the SCC.

```
static double H5SC__stats_get_hit_rate(const H5SC_t *cache);
```

Returns the SCC hit rate for a cache instance as a floating-point ratio (expected to be between 0.0 and 1.0). In this context, a hit indicates that a requested chunk lookup (within the SCC) successfully resolved to a resident non-placeholder chunk that was present within the cache.

```
static double H5SC__stats_get_miss_rate(const H5SC_t *cache);
```

Returns the SCC miss rate for a cache instance as a floating-point ratio (expected to be between 0.0 and 1.0). In this context, a miss indicates that a requested chunk lookup (within the SCC) did not resolve to a resident non-placeholder chunk that was present within the cache.

```
static herr_t H5SC__stats_dump(const H5SC_t *cache);
```

Function responsible for printing statistics tracked by the shared chunk cache. These statistics are stored in an `H5SC_stats_t` structure which is stored in `H5SC_t`.

7 Layout Callbacks

These are the callback functions that individual layout types implement in order to enable the shared chunk cache to perform these operations in a layout-agnostic manner. Some of these functions accept an `hsize_t *` parameter called `scaled`. This is an array containing the scaled coordinates of the chunk, where the coordinates are divided by the chunk dimensions so that adjacent chunks differ in their scaled coordinates by a value of one. In addition, in order for the shared chunk cache to calculate the logical locations of the chunks, the chunk dimensions will be promoted to the `H5D_shared_t` struct and made available for all layout types where it is valid (contiguous datasets will simply use the dataset dimensions). The layout will also need to specify whether it uses the legacy vlen/reference storage where data is stored outside the dataset, or whether the variable length data is stored in a separate section of the chunk. The contents of this section largely originated in [2] and have been updated/modified where appropriate.

```
typedef herr_t (*H5SC_chunk_lookup_t)(H5D_t *dset, size_t count, const
hsize_t *scaled[] /*in*/, haddr_t *addr[] /*out*/, hsize_t *size[] /*out*/,
```

```
hsize_t *defined_values_size[] /*out*/, size_t *size_hint[] /*out*/, size_t
*defined_values_size_hint[] /*out*/, void **udata[] /*out*/);
```

Looks up count chunk address and size on disk. `defined_values_size` is the number of bytes to read if only the list of defined values is needed. `size_hint` is the suggested allocation size for the chunk (could be larger if the chunk might expand when decoded). `defined_values_size_hint` is the suggested allocation size if only the list of defined values is needed. If `*defined_values_size` is returned as 0, then all values are defined for the chunk. In this case, the chunk may still be decoded without reading from disk, by allocating a buffer of size `defined_valued_size_hint` and passing it to `H5SC_chunk_decode_t` with `*nbytes_used` set to 0. `*udata` can be set to anything and will be passed through to `H5SC_chunk_decode_t` and/or the selection or vector I/O routines, then freed with `H5MM_xfree()` (we will create an `H5SC_free_udata_t` callback if necessary). Should the shared chunk cache determine that the chunk is not found on disk, the instance of `udata` referenced by this callback should be freed immediately.

```
typedef herr_t (*H5SC_chunk_decode_t)(H5D_t *dset, size_t *nbytes
/*in,out*/, size_t *alloc_size /*in,out*/, bool partial_bound, void **chunk
/*in,out*/, void *udata);
```

Decompresses/decodes the chunk from file format to memory cache format if necessary. Reallocs chunk buffer if necessary. On entry, `nbytes` is the number of bytes used in the chunk buffer. On exit, it shall be set to the total number of bytes used (not allocated) across all buffers for this chunk. On entry, `alloc_size` is the size of the chunk buffer. On exit, it shall be set to the total number of bytes allocated across all buffers for this chunk. Optional, if not present, chunk is the same in cache as on disk. `partial_bound` is true if the chunk was encoded with `partial_bound` set to true. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined.

```
typedef herr_t (*H5SC_chunk_decode_defined_values_t)(H5D_t *dset, size_t
*nbytes /*in,out*/, size_t *alloc_size /*in,out*/, bool partial_bound, void
**chunk /*in,out*/, void *udata);
```

The same as `H5SC_chunk_decode_t` but only decodes the defined values. Optional, if not present, the entire chunk must always be decoded.

```
typedef herr_t (*H5SC_new_chunk_t)(H5D_t *dset, bool fill, size_t *nbytes
/*out*/, size_t *buf_size /*out*/, void **chunk /*chunk*/, void **udata
/*out*/);
```

Creates a new empty chunk and a new empty `udata`. Does not insert into on disk chunk index. If `fill` is true, writes the fill value to the chunk (unless this is a sparse chunk). The number of bytes used is returned in `*nbytes` and the size of the chunk buffer is returned in `*buf_size`.

```
typedef herr_t (*H5SC_chunk_condense_t)(H5D_t *dset, size_t *nbytes /*in,
out*/, void **chunk /*in, out*/, void *udata);
```

Reallocates buffers as necessary so the total allocated size of buffers for the chunk (`alloc_size`) is equal to the total number of bytes used (`nbytes`). Optional, if not present the chunk cache will be more likely to evict chunks if there is wasted space in the buffers.

```
typedef herr_t (*H5SC_chunk_encode_t)(H5D_t *dset, hsize_t *write_size
/*out*/, hsize_t *write_buf_alloc /*out*/, bool partial_bound, const void
*chunk, void **write_buf /*out*/, void *udata);
```

Compresses/encodes the chunk as necessary. If chunk is the same as `cache_buf`, leaves `*write_buf` as `NULL`. This function leaves chunk alone and allocates `write_buf` if necessary to hold compressed data, sets `*write_size` to the size of the data in `write_buf`, and sets `*write_size_alloc` to the size of `write_buf`, if it was allocated. `partial_bound` is true if the chunk is partially outside the bounds of the dataset. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined.

```
typedef herr_t (*H5SC_chunk_encode_in_place_t)(H5D_t *dset, size_t
*write_size /*out*/, bool partial_bound, void **chunk /*in,out*/, void
*udata);
```

The same as `H5SC_chunk_encode_t` but does not preserve chunk buffer, encoding is performed in-place. Must free all other data used.

```
typedef herr_t (*H5SC_chunk_evict_t)(H5D_t *dset, void *chunk, void
*udata);
```

Frees chunk and all memory referenced by it. Optional, if not present `free()` is simply used.

```
typedef herr_t (*H5SC_chunk_insert_t)(H5D_t *dset, size_t count, const
hsize_t *scaled[] /*in*/, haddr_t *addr[] /*in,out*/, hsize_t
old_disk_size[], hsize_t new_disk_size[], void *udata[]);
```

Inserts (or reinserts) count chunks into the chunk index if necessary. Old address and size (if any) of the chunks on disk are passed as `addr` and `old_disk_size`, the new size is passed in as `new_disk_size`. This function resizes and reallocates on disk if necessary, returning the address of the chunks on disk in `*addr`.

```
typedef herr_t (*H5SC_chunk_selection_read_t)(H5D_t *dset, const H5S_t
*file_space_in, bool partial_bound, void *chunk /*in*/, H5S_t
**file_space_out /*out*/, bool *select_possible /*out*/, bool
*require_values /*out*/, void *udata);
```

Called when the chunk cache wants to read data directly from the disk to the user buffer via selection I/O. If not possible due to compression, etc, returns `*select_possible=false`. Otherwise transforms the file space if necessary to describe the selection in the on disk format (returns transformed space in `file_space_out`). If no transformation is necessary, leaves

`*file_space_out` as `NULL`. `chunk` may be passed as `NULL`, and may also be an in-cache chunk that only contains information on selected elements defined values. If `chunk` is passed as `NULL` and the callback requires a chunk to be passed with (at least) the defined values selection, this callback shall return `*require_values=true` and `*file_space_out=NULL`. Optional, if not present, chunk I/O is only performed on entire chunks or with vector I/O. The H5SC code checks for type conversion before calling this. `partial_bound` is true if the on-disk chunk was encoded with `partial_bound` set to true. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined.

```
typedef herr_t (*H5SC_chunk_vector_read_t)(H5D_t *dset, haddr_t addr, const
H5S_t *file_space_in, bool partial_bound, void *chunk /*in*/, size_t
*vec_count /*out*/, haddr_t **offsets /*out*/, size_t **sizes /*out*/, bool
*vector_possible /*out*/, bool *require_values /*out*/, void *udata);
```

Called when the chunk cache wants to read data directly from the disk to the user buffer via vector I/O. If not possible due to compression, etc., returns `vector_possible=false`. Otherwise returns the vector of selected elements in offsets (within the file, not the chunk, this is why `addr` is passed in) and sizes, with the number of vectors returned in `vec_count`. `chunk` may be passed as `NULL`, and may also be an in-cache chunk that only contains information on selected elements defined values. If `chunk` is passed as `NULL` and the callback requires a chunk to be passed with (at least) the defined values selection, this callback shall return `*require_values=true` and `*file_space_out=NULL`. Optional, if not present, chunk I/O is only performed on entire chunks or with selection I/O. The H5SC code checks for type conversion before calling this. `partial_bound` is true if the on-disk chunk was encoded with `partial_bound` set to true. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined. **-Not currently implemented.**

```
typedef herr_t (*H5SC_chunk_selection_write_t)(H5D_t *dset, const H5S_t
*file_space_in, bool partial_bound, void *chunk /*in*/, H5S_t
*file_space_out /*out*/, bool *select_possible /*out*/, bool
*require_values /*out*/, void *udata);
```

Called when the chunk cache wants to write data directly from the user buffer to the cache via selection I/O. If not possible due to compression, etc., returns `select_possible=false`. Otherwise transforms the file space if necessary to describe the selection in the on disk format (returns transformed space in `file_space_out`). If no transformation is necessary, leaves `*file_space_out` as `NULL`. `chunk` may be passed as `NULL`, and may also be an in-cache chunk that only contains information on selected elements defined values. If `chunk` is passed as `NULL` and the callback requires a chunk to be passed with (at least) the defined values selection, this callback shall return `*require_values=true` and `*file_space_out=NULL`. Optional, if not present, chunk I/O is only performed on entire chunks or with vector I/O. The H5SC code checks for type conversion before calling this. `partial_bound` is true if the on-disk chunk was encoded with `partial_bound` set to true. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined.

```
typedef herr_t (*H5SC_chunk_vector_write_t)(H5D_t *dset, haddr_t addr,
const H5S_t *file_space_in, bool partial_bound, void *chunk /*in*/, size_t
*vec_count /*out*/, haddr_t **offsets /*out*/, size_t **sizes /*out*/, bool
*vector_possible /*out*/, bool *require_values /*out*/, void *udata);
```

Called when the chunk cache wants to write data directly from the user buffer to the cache via vector I/O. If not possible due to compression, etc, returns `vector_possible=false`. Otherwise returns the vector of selected elements in offsets (within the file, not the chunk, this is why `addr` is passed in) and sizes, with the number of vectors returned in `vec_count`. `chunk` may be passed as `NULL`, and may also be an in-cache chunk that only contains information on selected elements defined values. If `chunk` is passed as `NULL` and the callback requires a chunk to be passed with (at least) the defined values selection, this callback shall return `*require_values=true` and `*file_space_out=NULL`. Optional, if not present, chunk I/O is only performed on entire chunks or with selection I/O. The H5SC code checks for type conversion before calling this. **-Not currently implemented.**

```
typedef herr_t (*H5SC_chunk_scatter_mem_t)(H5D_dset_io_info_t *dset_info,
H5D_io_type_info_t *io_type_info, const H5S_t *mem_space, const H5S_t
*file_space, const void *chunk, void *udata);
```

Scatters data from the chunk buffer into the memory buffer (in `dset_info`), performing type conversion if necessary. `file_space`'s extent matches the chunk dimensions and the selection is within the chunk. `mem_space`'s extent matches the entire memory buffer's and the selection within it is the selected values within the chunk, offset appropriately within the full extent. Optional, if not present, `chunk` is the same in memory as it is in cache, with the exception of type conversion (which will be handled by the H5SC layer). If the layout stores variable length data within the chunk this callback must be defined. `partial_bound` is true if the on-disk chunk was encoded with `partial_bound` set to true. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined.

```
typedef herr_t (*H5SC_chunk_gather_mem_t)(H5D_dset_io_info_t *dset_info,
H5D_io_type_info_t *io_type_info, const H5S_t *mem_space, const H5S_t
*file_space, size_t *nbytes /*in,out*/, size_t *alloc_size /*in,out*/,
size_t *bufalloc_size_total /*in,out*/, void *chunk, void *udata);
```

Gathers data from the memory buffer (in `dset_info`) into the chunk buffer, performing type conversion if necessary. `file_space`'s extent matches the chunk dimensions and the selection is within the chunk. `mem_space`'s extent matches the entire memory buffer's and the selection within it is the selected values within the chunk, offset appropriately within the full extent. Defines selected values in the chunk. `alloc_size_total` represents the total number of bytes allocated across all buffers for this chunk, and must be updated by this callback if that size changes. Optional, if not present, `chunk` is the same in memory as it is in cache, with the exception of type conversion (which will be handled by H5SC layer). If the layout stores variable length data within the chunk this callback must be defined.


```
typedef herr_t (*H5SC_chunk_fill_t)(H5D_dset_io_info_t *dset_info,
H5D_io_type_info_t *io_type_info, H5S_t *space, size_t *nbytes /*in,out*/,
size_t *alloc_size /*in,out*/, size_t *bufalloc_size_total /*in,out*/, void
*chunk, void *udata);
```

Propagates the fill value into the selected elements of the chunk buffer, performing type conversion if necessary. `space`'s extent matches the chunk dimensions and the selection is within the chunk. `alloc_size_total` represents the total number of bytes allocated across all buffers for this chunk, and must be updated by this callback if that size changes. Optional, if not present, chunk is the same in memory as it is in cache, with the exception of type conversion (which will be handled by H5SC layer). If the layout stores variable length data within the chunk this callback must be defined.

```
typedef herr_t (*H5SC_chunk_defined_values_t)(H5D_t *dset, const H5S_t
*selection, void *chunk, void *udata, H5S_t **defined_values /*out*/);
```

Queries the defined elements in the chunk. `selection` may be passed as `H5S_ALL`. These selections are within the logical chunk. Optional, if not present, all values are defined.

```
typedef herr_t (*H5SC_chunk_erase_values_t)(H5D_t *dset, const H5S_t
*selection, size_t *nbytes /*in,out*/, size_t *alloc_size /*in,out*/, void
*chunk, void *udata, bool *delete_chunk /*out*/);
```

Erases the selected elements in the chunk, causing them to no longer be defined. If all values in the chunk are erased and the chunk should be deleted, sets `*delete_chunk` to `true`, causing the cache to delete the chunk from cache, free it in memory using `H5SC_chunk_evict_t`, and delete it on disk using `H5SC_chunk_delete_t`. These selections are within the logical chunk. Optional, if not present, the fill value will be written to the selection using `H5SC_chunk_fill_t`.

```
typedef herr_t (*H5SC_chunk_evict_values_t)(H5D_t *dset, size_t *nbytes
/*in,out*/, size_t *alloc_size /*in,out*/, void *chunk, void *udata);
```

Frees the data values in the cached chunk and memory used by them (but does not reallocate – see `H5SC_chunk_condense_t`), but leaves the defined values intact. Optional, if not present the entire chunk will be evicted.

```
typedef herr_t (*H5SC_layout_query)(H5D_t *dset, hsize_t *chunk_dims, bool
*encode_decode_necessary, bool *partial_bound_chunks_different_encoding);
```

Queries data about the dataset from the layout client. The callback shall set the chunk dimensions in the `chunk_dims` array (the number of dimensions is the same as the rank of the dataset), whether encoding and decoding is necessary for chunks between cache and disk, and shall set whether chunks that are partially outside the bounds of the dataset are encoded differently (for example, they may not have filters applied). If `*partial_bound_chunks_different_encoding` is set to `true`, then chunks whose partial bound state changes will be re-encoded and re-inserted as necessary after the dataset extent changes to ensure they are encoded appropriately.


```
typedef herr_t (*H5SC_delete_chunk_t)(H5D_t *dset, const hsize_t *scaled
/*in*/, haddr_t addr, hsize_t disk_size);
```

Removes the chunk from the index and deletes it on disk. Only called if a chunk goes out of scope due to `H5Dset_extent()` or if `H5SC_chunk_erase_values_t` returns `*delete_chunk == true`.

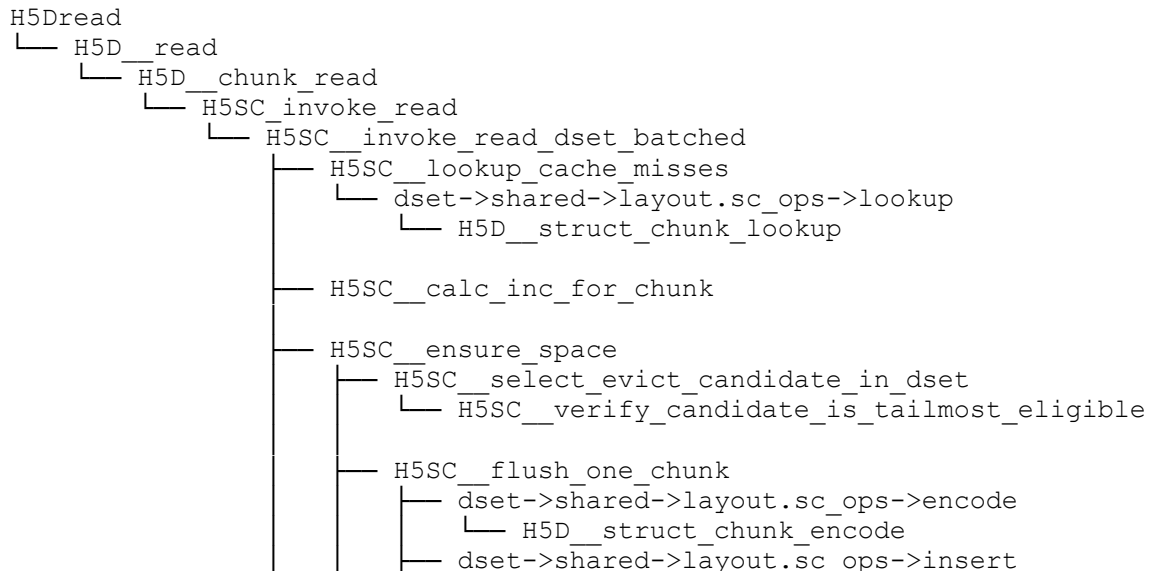
7.1.1 Callback struct

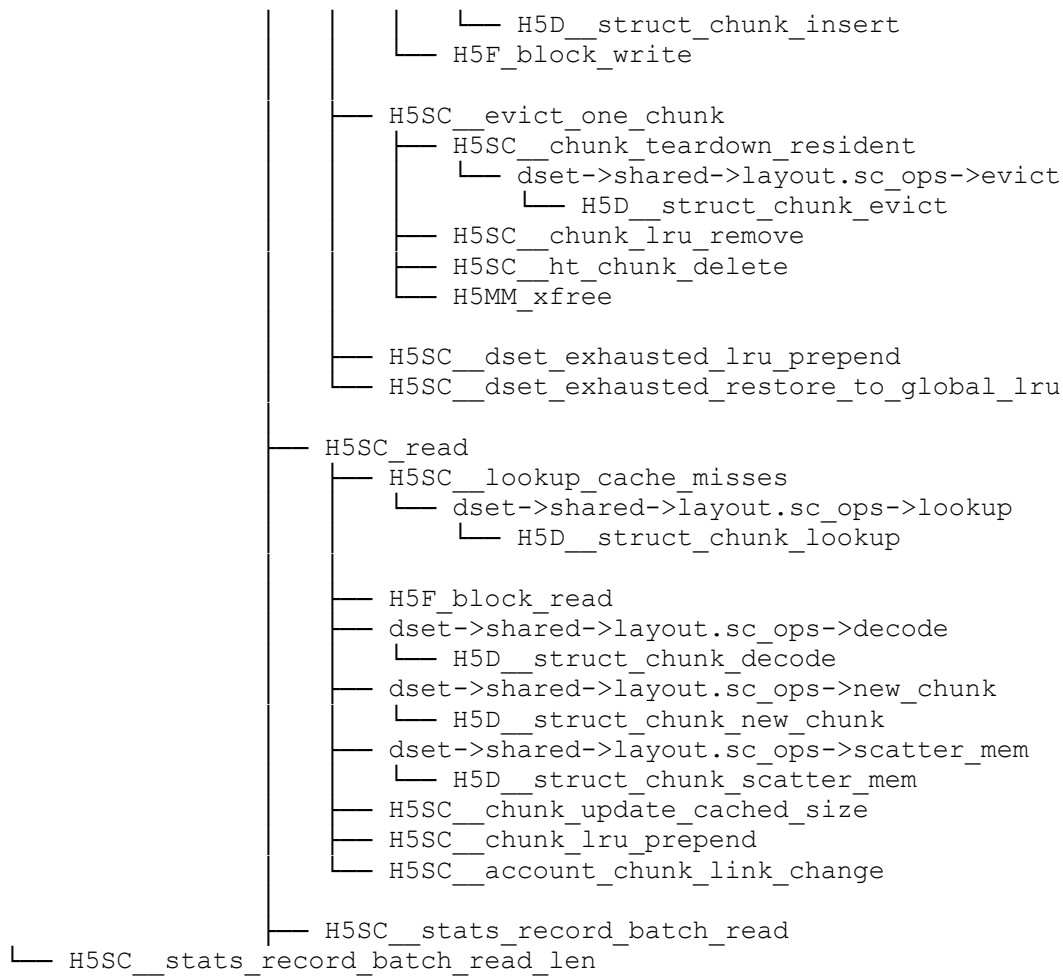
The final callback structure for each layout class is therefore:

```
typedef H5SC_layout_ops_t {
H5SC_chunk_lookup_t lookup;
H5SC_chunk_decode_t decode;
H5SC_chunk_decode_defined_values_t decode_defined_values;
H5SC_new_chunk_t new_chunk;
H5SC_chunk_condense_t condense;
H5SC_chunk_encode_t encode;
H5SC_chunk_evict_t evict;
H5SC_chunk_encode_in_place_t encode_in_place;
H5SC_chunk_insert_t insert;
H5SC_chunk_selection_read_t selection_read;
H5SC_chunk_vector_read_t vector_read;
H5SC_chunk_selection_write_t selection_write;
H5SC_chunk_vector_write_t vector_write;
H5SC_chunk_scatter_mem_t scatter_mem;
H5SC_chunk_gather_mem_t gather_mem;
H5SC_chunk_fill_t fill;
H5SC_chunk_defined_values_t defined_values;
H5SC_chunk_erase_values_t erase_values;
H5SC_chunk_evict_values_t evict_values;
H5SC_layout_query layout_query;
H5SC_delete_chunk_t delete_chunk;
} H5SC_layout_ops_t;
```

8 Call Tree Diagrams

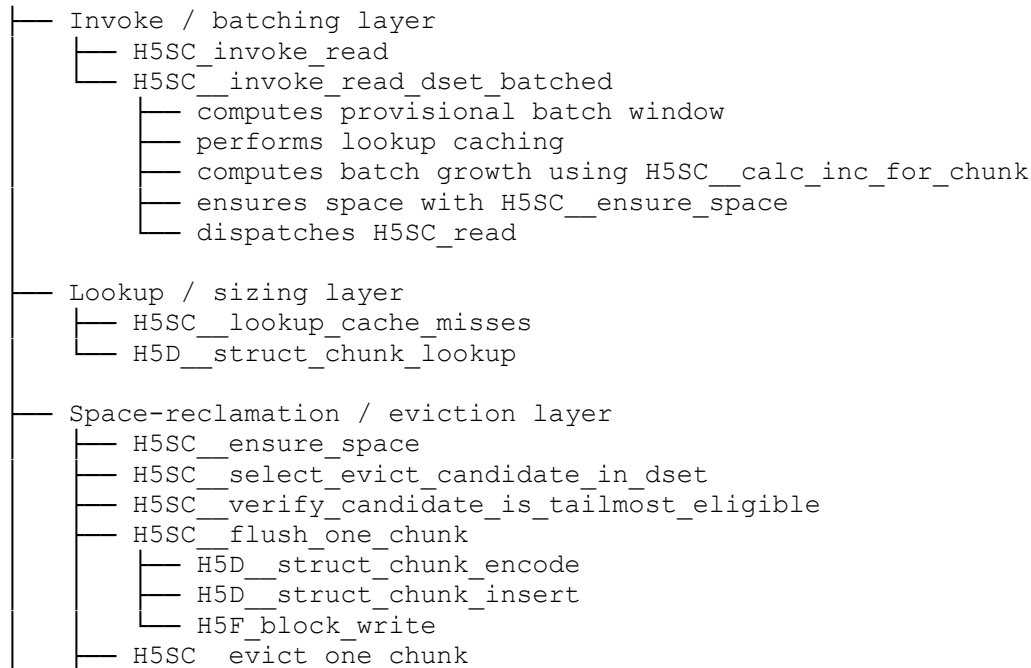
8.1. Reading data through `H5Dread()`:

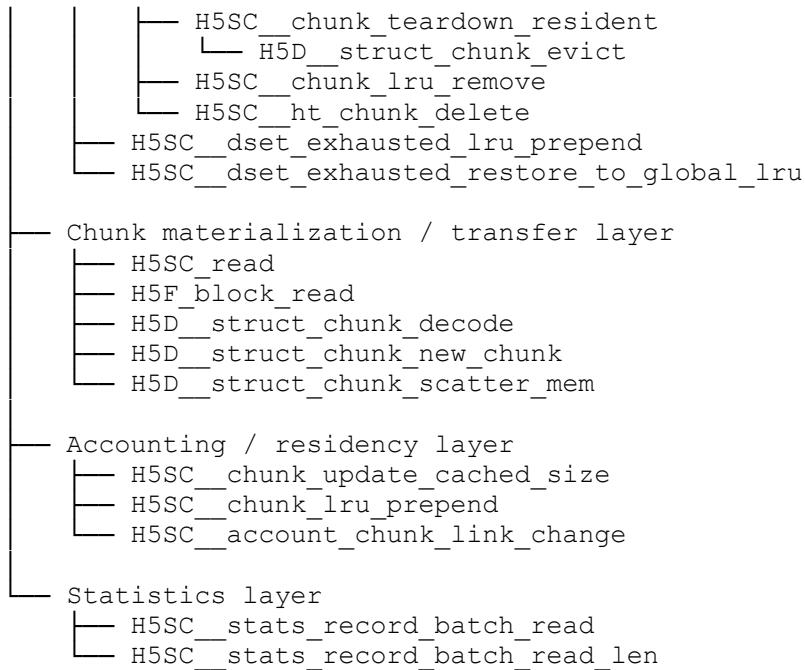




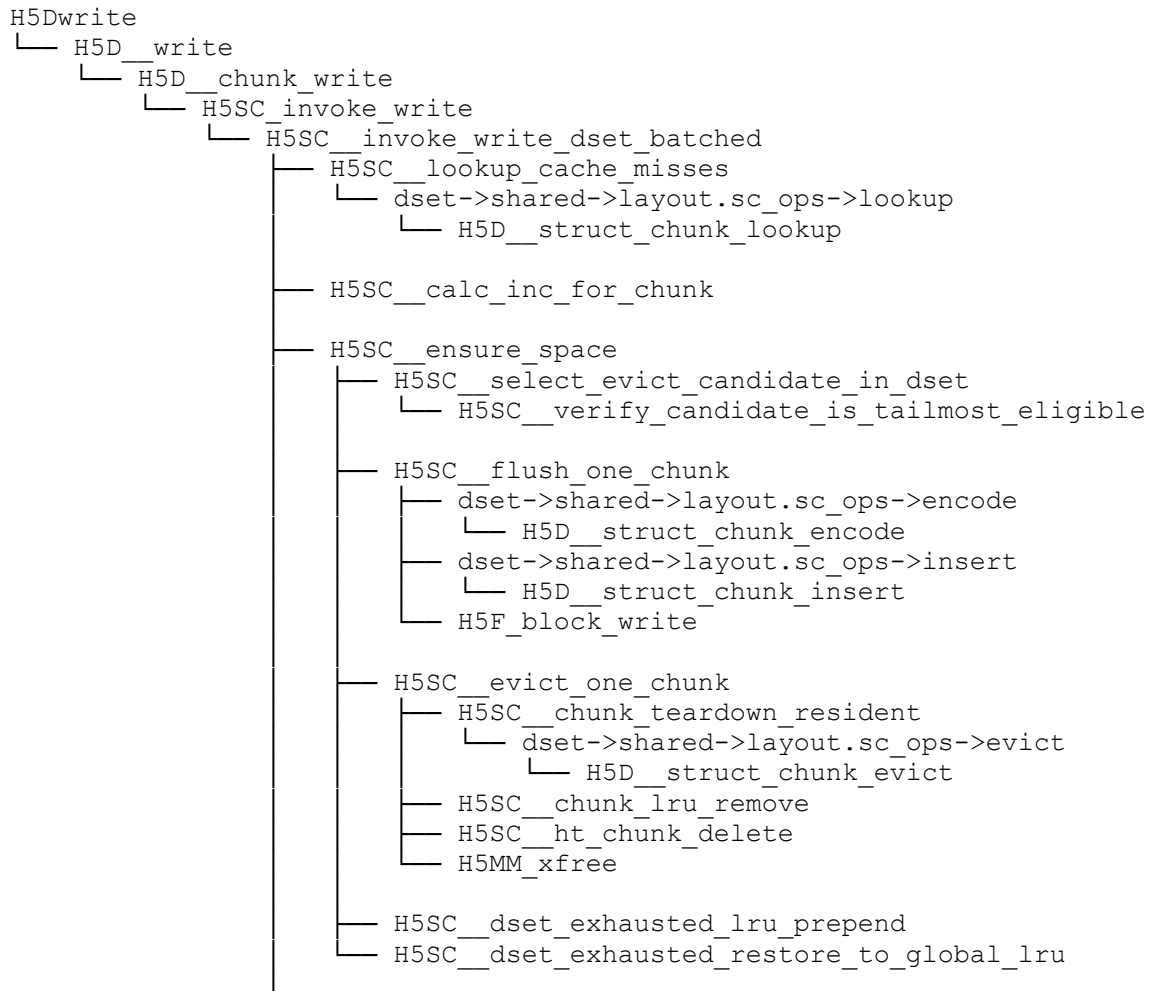
8.1.1 Data Read Call Tree by Subsystem:

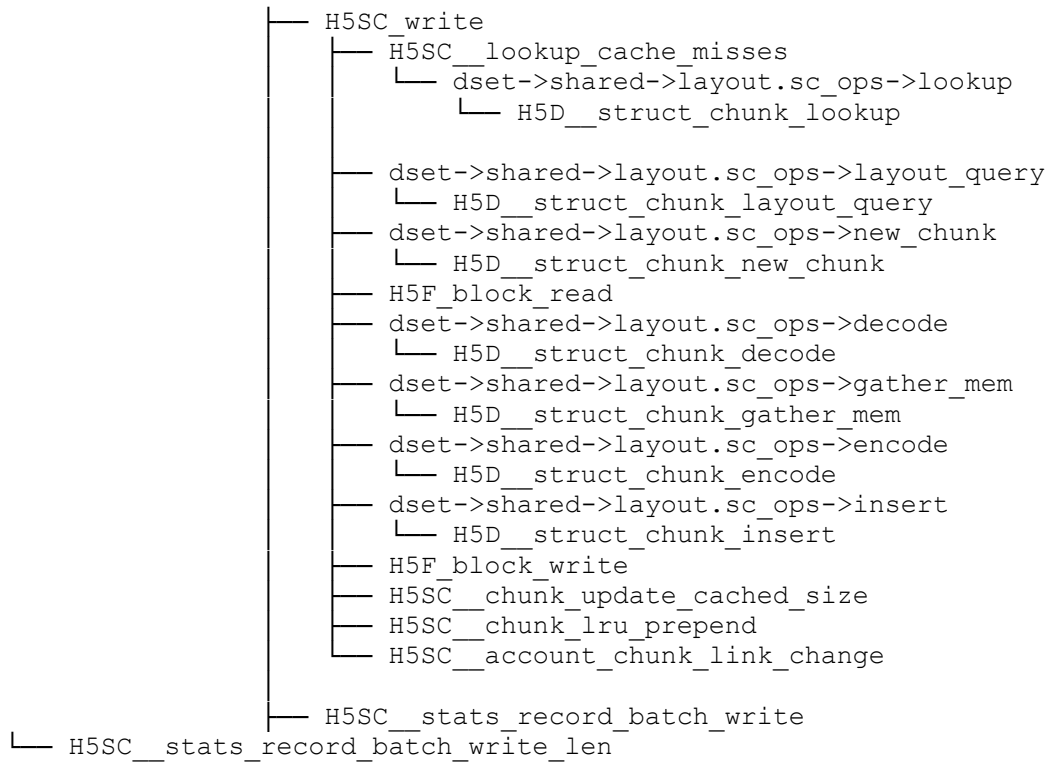
Read Path





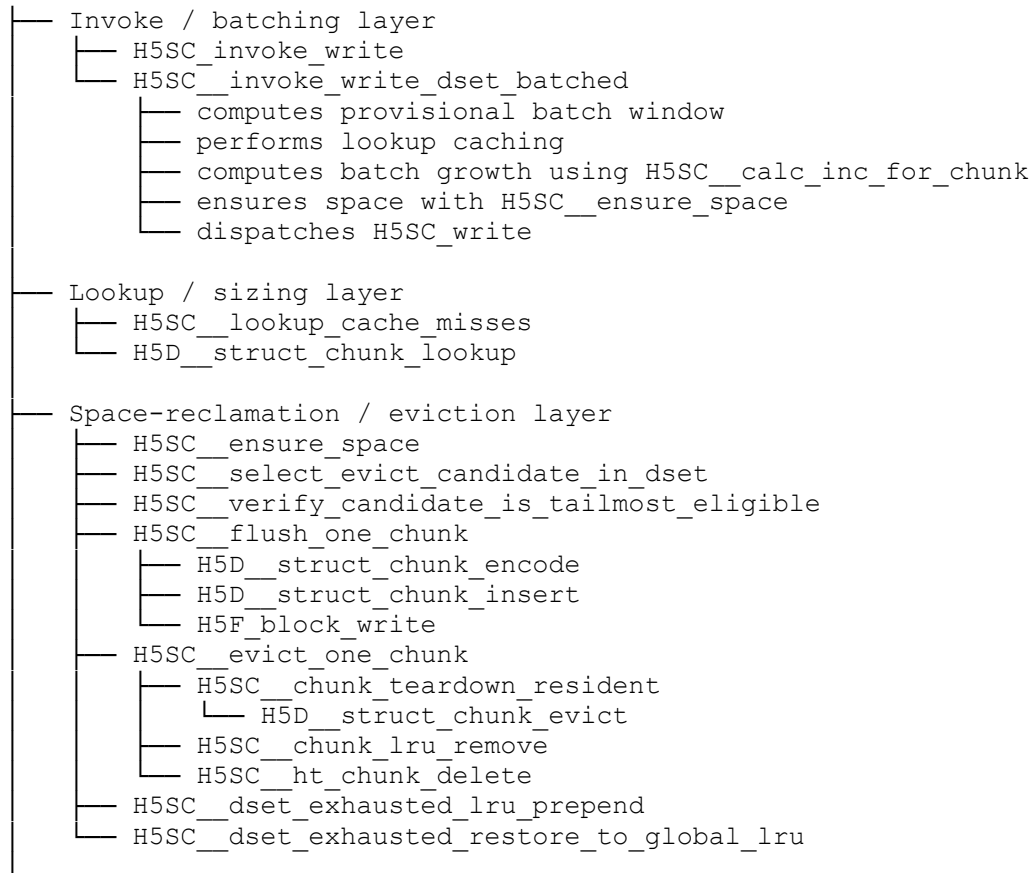
8.2. Writing data through H5Dwrite():

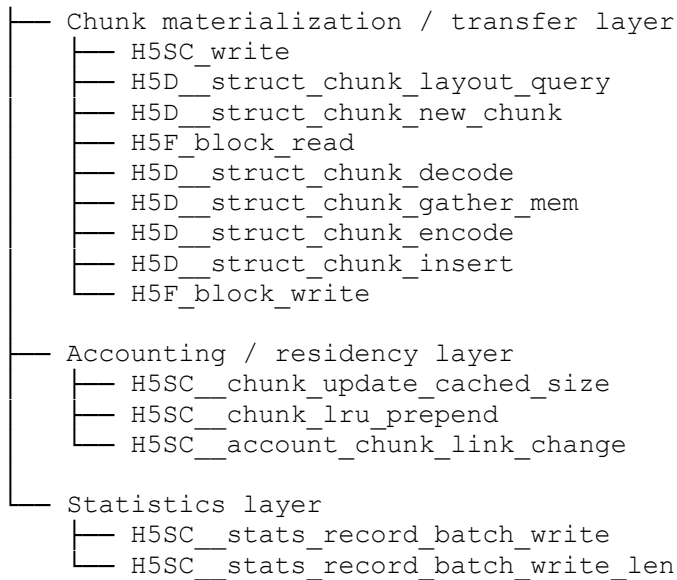




8.2.1 Data Write Call Tree by Subsystem:

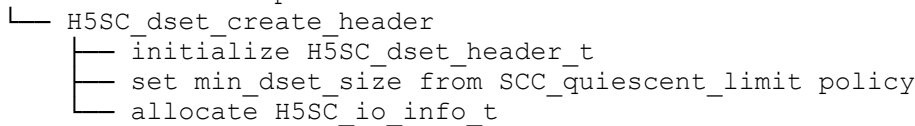
Write Path





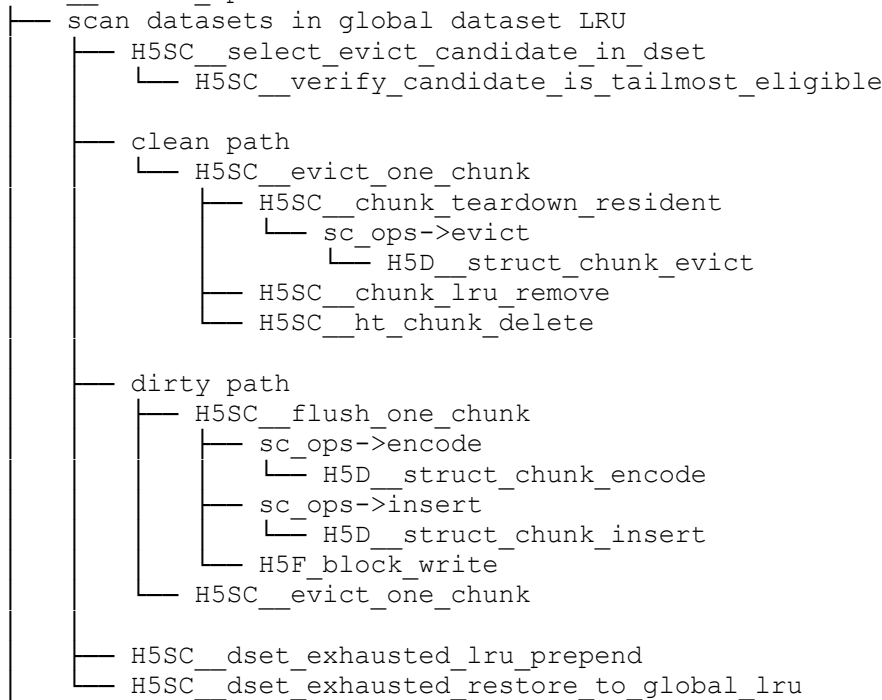
8.3. Supporting Initialization and Dataset Setup

Dataset creation path



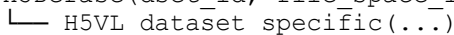
8.4. Eviction and Flush Subsystems

H5SC__ensure_space



8.5. H5Derase() Functionality

H5Derase(dset_id, file_space_id, dxpl_id)



```

└─ H5VL_dataset_specific(...)
  └─ H5VL_native_dataset_specific(...)
    └─ H5D_erase(dset, fspace)
      └─ if dset->shared->layout.sc_ops exists
        └─ H5SC_erase(H5F_SHARED_CACHE(dset->oloc.file), dset, fspace)
          └─ H5SC_erase_io_info_init(...)
            └─ reset SCC I/O info
            └─ compute selected chunks from file selection
            └─ build chunk-local erase selections
            └─ find/create SCC chunk entries
            └─ pin selected chunks

            └─ for each selected chunk
              └─ if chunk is not resident
                └─ if structured storage unallocated
                  └─ unpin and skip
                └─ sc_ops->lookup(...)
                └─ if sparse miss
                  └─ unpin and skip
                └─ H5F_block_read(...)
                └─ sc_ops->decode(...)
                  └─ H5D_struct_chunk_decode(...)

                └─ if defined_values callback exists
                  └─ sc_ops->defined_values(...)
                    └─ H5D_struct_chunk_defined_values(...)

                └─ sc_ops->erase_values(...)
                  └─ H5D_struct_chunk_erase_values(...)

                └─ if delete_chunk
                  └─ sc_ops->delete_chunk(...)
                    └─ H5D_struct_chunk_delete_chunk(...)
                  └─ H5SC_chunk_lru_remove(...)
                  └─ H5SC_chunk_tear_down_resident(...)
                  └─ H5SC_ht_chunk_delete(...)
                  └─ free H5SC_chunk_t

                └─ else chunk survives
                  └─ mark dirty
                  └─ H5SC_flush_one_chunk(...)
                    └─ sc_ops->encode(...)
                      └─ H5D_struct_chunk_encode(...)
                    └─ sc_ops->insert(...)
                      └─ H5D_struct_chunk_insert(...)
                    └─ H5F_block_write(...)
                  └─ unpin chunk

              └─ H5SC_account_chunk_link_change(...)
              └─ H5SC_io_info_reset(...)

            └─ else
              └─ error: dataset does not support erasing values

```

8.6. H5Dset_extent Functionality

```

H5Dset_extent(dset_id, size)
└─ H5D_set_extent_api_common(...)
  └─ H5D_set_extent(dset, size)
    └─ save old_dims

```

```

└─ update dataspace extent
└─ update layout / chunk indexing metadata
└─ if dset->shared->layout.sc_ops exists
    └─ H5SC_set_extent_notify(cache, dset, old_dims)
        └─ read current extent into ext_dims
        └─ update dset->shared->curr_dims
        └─ H5D_struct_chunk_set_info(dset)
            └─ update structured-chunk indexing metadata
        └─ if extent changed and structured storage allocated
            └─ H5SC_prune_by_extent(cache, dset, old_dims)
                └─ query chunk geometry / encoding policy
                    └─ sc_ops->layout_query(...)
                └─ compute old_nchunks / new_nchunks
                └─ determine shrink / growth / chunk-grid change
                └─ if growth only
                    └─ if chunk-grid changed
                        └─ H5SC_rekey_dset_chunks_after_extent_change(...)
                            └─ remove dataset chunks from SCC hash
                            └─ recompute chk_log_coord / data_key
                            └─ reinsert chunks into SCC hash
                └─ if shrink
                    └─ Pass 1: traverse cached SCC chunks
                        └─ Case 1: chunk fully outside new extent
                            └─ sc_ops->delete_chunk(...)
                                └─ H5D_struct_chunk_delete_chunk(...)
                            └─ sc_ops->evict(...)
                                └─ H5D_struct_chunk_evict(...)
                            └─ H5SC_chunk_lru_remove(...)
                            └─ H5SC_ht_chunk_delete(...)
                            └─ free H5SC_chunk_t
                        └─ Case 2: surviving chunk overlaps invalid region
                            └─ H5SC_erase_chunk_invalid_extent_region(...)
                                └─ compute old_valid_count / new_valid_count
                                └─ build chunk-local invalid-region slabs
                                └─ sc_ops->erase_values(...)
                                    └─ H5D_struct_chunk_erase_values(...)
                        └─ Case 3: erase emptied chunk
                            └─ sc_ops->delete_chunk(...)
                                └─ H5D_struct_chunk_delete_chunk(...)
                            └─ sc_ops->evict(...)
                                └─ H5D_struct_chunk_evict(...)
                            └─ H5SC_chunk_lru_remove(...)
                            └─ H5SC_ht_chunk_delete(...)
                            └─ free H5SC_chunk_t
                        └─ Case 4: surviving dirty/re-encoded chunk
                            └─ H5SC_flush_one_chunk(...)
                                └─ sc_ops->encode(...)
                                    └─ H5D_struct_chunk_encode(...)
                                └─ sc_ops->insert(...)
                                    └─ H5D_struct_chunk_insert(...)
                                └─ H5F_block_write(...)
                    └─ Pass 2: storage-only whole-chunk cleanup
                        └─ H5SC_for_each_scaled_coord_outside_extent(...)
                        └─ H5SC_prune_one_scaled_coord_outside_extent(...)

```

```

└─ H5SC__lookup_chunk_for_prune(...)
    └─ sc_ops->lookup(...)
    └─ sc_ops->delete_chunk(...)
        └─ H5D__struct_chunk_delete_chunk(...)

if chunk-grid changed
└─ H5SC__rekey_dset_chunks_after_extent_change(...)
    └─ remove dataset chunks from SCC hash
    └─ recompute chk_log_coord / data_key
    └─ reinsert chunks into SCC hash

H5SC__account_chunk_link_change(...)

```

9 Test Coverage Summary

The current H5SC test suite, provided in `./hdf5/test/shared_chunk_cache.c`, provides broad coverage across both the low-level SCC data structures and the higher-level structured-chunk I/O paths. Storage-specific testing is provided in `./hdft/test/struct_chunk_storage.c`. At the internal-component level, the suite exercises logical chunk index computation, chunk-key construction, bit interleaving/deinterleaving¹², dataset and chunk hash tables, dataset-level and per-dataset chunk LRUs, accounting invariants, list splice helpers, lookup helpers, and lifecycle/state-tracking behavior. The logical-index tests validate chunk-grid computations across multiple ranks, including explicit verification of `nchunks[]`, `down[]` multiplier generation, scaled-coordinate bounds checking, and C-order linear chunk index computation against independent reference implementations.

The chunk-key tests provide deterministic and randomized validation of the interleaved 128-bit key generation logic used by SCC chunk identity tracking. These tests verify both individual bit placement and full encode/decode round-trip correctness using randomized fuzz-style input generation. Internal DLL and LRU tests exercise prepend/remove behavior for both dataset-header LRUs and per-dataset chunk LRUs while explicitly validating list ordering, length counters, and resident-byte accounting updates.

The suite also includes lifecycle-oriented regression tests that validate structure integrity and SCC operation tagging across allocation, hash insertion, LRU insertion, flush processing, and lookup revalidation. These tests verify preservation of magic values, pointer identity, and operation-state tracking fields as SCC objects transition through the cache-management pipeline. Collectively, these low-level tests provide strong coverage of SCC bookkeeping correctness, list integrity, hash-table behavior, accounting consistency, key generation, and structural state propagation before higher-level dataset I/O paths are exercised.

At the structured-chunk dataset level, the suite covers the major SCC operational paths through a combination of deterministic correctness tests and large integration-style stress tests. Basic 1D and 2D tests validate single-chunk processing, `H5S_ALL` reads, explicit hyperslab selections, multi-dataset isolation, reopen persistence, and structured chunk dataset creation. The FAPL configuration path is

¹² Deinterleaving is not currently used with in the functionality of the H5SC module.

exercised through SCC cache configuration APIs, and multiple tests intentionally use aggressive SCC limits to force eviction, batching, quiescent trimming, and cross-dataset cache turnover.

Extent and erase coverage is substantial. The suite exercises `H5Derase()` over full chunks, partial chunks, undefined chunks, rewritten chunks, and mixed overwrite/erase patterns across multiple ranks. Extent tests cover shrink pruning, storage-only pruning, shrink/regrow cycles, reopen persistence, one and two extensible dimensions, and 3D datasets with multiple changing dimensions. The shrink-prune tests explicitly validate that whole chunks outside the new extent are removed while surviving chunks retain correct data and SCC accounting state. These tests collectively validate synchronization between SCC cache state, structured chunk storage state, and visible dataset contents during complex extent transitions.

Filter coverage is also extensive. `GZIP/DEFLATE` paths are exercised through fixed and selection sections, sparse point-like hyperslab updates, `H5S_ALL` I/O, multidimensional datasets, erase flows, reopen verification, and mixed filter stacks including `SHUFFLE + GZIP` and `FLETCHER32 + GZIP`. `SZIP` coverage now includes `NN/EC` coding modes, multiple pixels-per-block configurations, sparse selections, reopen persistence, erase, shrink/regrow, and mixed-section filtering scenarios where selection metadata uses `GZIP` while fixed payload data uses `SZIP`. The tests also intentionally document current limitations, such as unsupported `SZIP` application to selection metadata sections.

The broadest end-to-end validation is the mixed-rank eviction stress suite. This test exercises seven datasets simultaneously, spanning ranks 1D through 5D and combining extensible and non-extensible datasets, filtered and unfiltered datasets, `H5S_ALL` I/O, sparse updates, erase, shrink/regrow, reopen verification, `SZIP` filtering, mixed-section filtering, and intentionally constrained SCC limits designed to force aggressive eviction and dataset turnover. The resulting workload provides meaningful integration coverage for SCC accounting, eviction policy, dataset retention rules, and multi-dataset cache interactions under sustained pressure.

10 Multi-threaded support

This section outlines modifications to support multi-thread operations in SCC and external support code required for multi-thread operation of the shared chunk cache.

10.1 Multi-threaded operations in SCC

The following tasks will be performed to support multi-threading in SCC:

1. Switch to multi-thread safe data structures
 - a. Use lock free hash table instead of `UThash` for the chunk and dataset hash tables,
 - b. Replace the doubly linked lists used for the per data set chunk LRUs and the dataset LRU with a lock free equivalent TBD.
2. Modify existing structures to use atomic values where appropriate -- i.e., lengths of and sizes of lists, etc.
3. Code modifications to support the above.

4. While not immediately necessary, for efficiency, we should support concurrent decoding of chunks that have already been read from disk, and encoding of chunks that are about to be flushed and potentially evicted. We need to implement a thread pool to support this functionality and modify the lower-level callbacks as needed to ensure thread safety.

10.2. Additional code for supporting multi-thread operations in SCC

This sub-section outlines external support code required for multi-thread operation of the shared chunk cache.

Serialization module: When I/O requests are serviced in parallel, we must ensure that only overlapping reads are executed concurrently. Overlapping writes, and any intervening overlapping reads must be serialized. The serialization module examines the stream of incoming I/O requests, and delays passing them to the shared chunk cache as necessary to ensure this. When changing a dataset's extent, we drain all current I/O requests and then marks the dataset as locked until the resizing operation is completed.

Resource allocation module: Large I/O requests can be many times the size of the shared chunk cache. Such I/O requests will require exclusive access to the shared chunk cache if we are to avoid thrashing. More generally, we must estimate I/O request resource requirements and only allow concurrent requests into the shared chunk cache if it has sufficient resources to serve the requests without thrashing. The resource allocation module exists to enforce this.

Multi-thread support in the VFD layer: Concurrent service of multiple I/O requests in the shared chunk cache requires the ability to support concurrent I/O requests in the VFD layer.

Other considerations: Pending multi-thread support in the chunk indices and the metadata cache, we may need to add additional serialization to I/O requests to avoid corruption caused by interleaved modifications to the chunk indexes. This shouldn't be an issue with concurrent reads, however, concurrent writes to the same dataset have potential issues, as do any reads of the dataset concurrent with a write.

The exact potential issues here require further investigation, and it may be that there are no issues not solved by the global mutex.

11 Acknowledgement

This work is supported by the U.S. Department of Energy, Office of Science under award number DE-SC0023583 for Phase I and Phase II SBIR project "Supporting Sparse Data in HDF5"

12 Appendix A: SCC Version 0 Implementation Description

The Shared Chunk Cache, Version 0 (V0), is a minimally functional representation of how structured chunk data will be processed through the cache. This initial version focuses on demonstrating the connection between the high-level API (through `H5D__write()`) and the low-level callbacks (which

are invoked by the SCC). To maintain a simplistic, verifiable output, a single chunk within a single dataset was initially the sole focus of this development, with the end goal being supporting I/O on multiple chunks within a single dataset. To illustrate proper cache functionality, structured chunk-formatted data would first need to be written to a file and then read back. In V0, the cache simply acts as a pass-through for the data buffer between the high-level API and the low-level callbacks, effectively simulating the action of caching the data (i.e. in this instance is assumed that the cache can only “hold” a single chunk; any additional operations involving the cache would require eviction). This approach serves two primary purposes. First, it was necessary to test both the high-level and low-level code integration, which up to this point, had remained untested (by necessity). Second, the V0 cache would provide the developer further exposure to essential internal HDF5 components that would further inform this redesign. With these goals in mind, the V0 development initially focused on creating, writing, and reading a 1-dimensional dataset with five elements and a single chunk of the same size. For simplicity, the chunk would contain non-fill values at indexes 1, 2, and 3 to provide a quick, consistent point of reference.

To facilitate testing the high- and low-level code integration, the write process was the first to be focused on, through the development of the `H5SC_write()` function. Using chunk information derived from first calling `H5SC__io_info_init()`, two key tasks are done; first the caching action for a dirty chunk is emulated, followed by a simulation of the eviction process. Upon calling `H5SC_write()`, `H5SC__io_info_init()` is called, which processes selections for each chunk in the I/O request. Once this preliminary chunk information is collected, the chunk is looked up on disk (which, by assumption, will result in a failure to find the chunk). Since the chunk is not found on disk, it should be fully overwritten, which leads to invoking the new chunk callback to initialize a new chunk. After the new chunk is created, the gather memory callback is called to collect the chunk data into a single buffer. Next, the "eviction" process begins. First, the chunk is encoded to ensure that it is transitioned from the in-memory format into the on-disk format. Then, the chunk is inserted into the file indexing using the insert callback function. Finally, the encoded chunk buffer is written to disk using the `H5F_block_write()` function. On-disk data verification was done using the Unix octal dump (`od`) tool.

After the write process was debugged and verified, the next step was to read the data back into a user buffer through developing the `H5SC_read()` function. Similar to the procedure used to write the initial chunk, `H5SC__io_info_init()` is used to collect necessary information about the requested chunk. Using this information, the chunk is looked up (and this time around, it is expected to be present). If the chunk is found on disk, the chunk read process is initiated. Using the information collected with the lookup callback, `H5F_block_read()` is then invoked to load the on-disk formatted chunk into a buffer. This buffer is then run through the decode callback, which translates the data into the in-memory format. Finally, this buffer is scattered into the available memory using the scatter memory callback.

Once this process was completed for a single chunk in a single dataset, it was then extended to support writes and reads on a single dataset with multiple chunks. During this process, the callbacks were tested further and adjusted to ensure proper functionality when an I/O request necessitates processing multiple chunks.

13 **References**

1.

John Mainzer, Elena Pourmal, “RFC: New Requirements for HDF5 Chunk Cache”,
https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/RFC-HDF5-Chunk-Cache-Req-2023-12-18.pdf.

2.

Neil Fortner, John Mainzer, Vailin Choi, Clay Carper, “RFC: Shared Chunk Cache Internal API”,
https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/Shared_Chunk_Cache_API.v11.pdf

3.

Hanson, Troy. “Uthash: A Hash Table for c Structures.” *Github.io*, 2012,
troydhanson.github.io/uthash/.

Revision History

<i>July 25, 2025:</i>	Version 0 sent for internal review.
<i>July 31, 2025:</i>	Version 1 sent for internal review.
<i>August 27, 2025</i>	Version 2-8 copyediting; 6 version added new section of multi-threaded updates.
<i>April 8, 2026</i>	Version 9 updated to reflect Version 1 development and changes; began integrating the content of other design documents for cohesion and ease of access.
<i>April 22, 2026</i>	Version 10 provided further updates to reflect Version 1 development and changes.
<i>May 19, 2026</i>	Version 11 includes updates to reflect Version 1 development, including filter support and testing coverage.